



Tribhuvan University
Institute of Science and Technology

Project Report on
InfoDigest – Text Summarizer

Submitted to:

Department of Bachelor of Science in Computer Science and Information
Technology (BSc.CSIT)
National College of Computer Studies

*In partial fulfillment of the requirement for the degree of Bachelor of
Science in Computer Science and Information Technology (BSc.CSIT)*

Submitted by:

Mandish Nanda Vaidya [28872/078]
Regd. No.: 5-2-551-76-2021

Under the supervision of

Mr. Yuba Raj Devkota

August, 2025



NATIONAL COLLEGE OF COMPUTER STUDIES

TRIBHUVAN UNIVERSITY

SUPERVISOR’S RECOMMENDATION

I hereby recommend that this project, prepared under my supervision entitled **“InfoDigest – Text Summarizer”**, a platform that performs extractive and abstractive summarization, by **Mandish Nanda Vaidya** in partial fulfillment of the requirements for the degree of Bachelor of Science in Computer Science and Information Technology (BSc.CSIT) is processed for the evaluation.

.....

Mr. Yuba Raj Devkota

Project Supervisor

National College of Computer Studies

Paknajol, Kathmandu

CERTIFICATE OF APPROVAL

ACKNOWLEDGEMENT

The successful completion of my final year project owes much to the invaluable support and guidance extended by my project supervisors. I express my deepest gratitude to **Mr. Yuba Raj Devkota**, my project supervisor, whose dedicated guidance and mentorship have been instrumental throughout my journey. I am equally grateful to **Mr. Jagdish Bhatta** from Tribhuvan University for providing us with a comprehensive project overview and direction.

I would like to express my heartfelt thanks to the National College of Computer Studies (NCCS) for providing me with an exceptional platform to pursue and develop this project. Under the guidance of **Mr. Yuba Raj Devkota**, and the NCCS team, I have significantly deepened my understanding of AI and ML algorithms, as well as various components associated with the development of this system.

I am profoundly grateful for the unwavering support and encouragement I received throughout this academic endeavor. The NCCS administration, faculty members, and technical staff have been exceptional in providing the resources and environment conducive to my research and development. The collective contribution of my friends and communities has not only enhanced the quality of my project but also enriched my learning experience, making this journey truly memorable and worthwhile.

ABSTRACT

InfoDigest is an AI-powered web application designed to combat information overload by intelligently condensing large volumes of text from scientific and research documents. Leveraging both extractive and abstractive techniques, the system automatically generates coherent summaries from various document formats, including .docx, .txt, and .pdf. Beyond its core summarization capabilities, the platform integrates essential text analysis metrics such as word count and keyword extraction.

The system features a user-friendly interface and a robust authentication framework that differentiates between standard users and administrators. Authenticated users can access their summary history and provide feedback, while administrators manage users and view overall system analytics. This integrated approach streamlines the research workflow, making academic information more accessible and digestible for a wide range of users.

KEYWORDS: *Extractive summarization, Abstractive summarization, Document summarization, Natural Language Processing, Text rank, LongT5*

TABLE OF CONTENTS

SUPERVISOR’S RECOMMENDATION	i
CERTIFICATE OF APPROVAL	ii
ACKNOWLEDGEMENT.....	iii
ABSTRACT.....	iv
LIST OF ABBREVIATIONS	vii
LIST OF FIGURES	viii
LIST OF TABLES.....	ix
CHAPTER 1: INTRODUCTION.....	1
1.1 Introduction.....	1
1.2 Problem Statement	1
1.3 Objectives	2
1.4 Scope and Limitation	2
1.4.1. Scope.....	2
1.4.2. Limitations	2
1.5 Development Methodology	3
1.6 Report Organization.....	4
CHAPTER 2: BACKGROUND STUDY AND LITERATURE REVIEW.....	5
2.1 Background Study.....	5
2.2 Literature Review.....	5
CHAPTER 3: SYSTEM ANALYSIS.....	8
3.1 System Analysis	8
3.1.1 Requirement Analysis	8
3.1.2 Feasibility Analysis.....	10
3.1.3 Analysis.....	11
CHAPTER 4: SYSTEM DESIGN	17

4.1 Design	17
4.1.1 Refinement of Class, Object, State, Sequence and Activity diagrams.....	17
4.1.2 Component diagrams	21
4.1.3 Deployment diagrams	22
4.2 Algorithm Details.....	22
4.2.1 Extractive Summarization.....	22
4.2.2 Abstractive algorithm.....	25
CHAPTER 5: IMPLEMENTATION AND TESTING.....	29
5.1 Implementation	29
5.1.1. Tools Used.....	29
5.1.2. Implementation Details of Modules.....	29
5.2. Testing.....	41
5.2.1. Test Cases for Unit Testing	41
5.2.2. Test Cases for System Testing.....	44
5.3. Result Analysis.....	46
CHAPTER 6: CONCLUSION AND FUTURE RECOMMENDATIONS	49
6.1. Conclusion	49
6.2. Future Recommendations	49
REFERENCES.....	50
APPENDICES	51

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
API	Application Programming Interface
BART	Bidirectional and Auto-Regressive Transformers
BLEU	Bilingual Evaluation Understudy
CNN	Convolutional Neural Network
GPT	Generative Pre-trained Transformer
GPU	Graphics Processing Unit
IDE	Integrated Development Environment
LSA	Latent Semantic Analysis
LSTM	Long Short-Term Memory
MERN	MongoDB, Express, React, Node.js
ML	Machine Learning
NLP	Natural Language Processing
PDF	Portable Document Format
RAG	Retrieval-Augmented Generation
ROGUE	Recall-Oriented Understudy for Gisting Evaluation
RNN	Recurrent Neural Network
Seq2Seq	Sequence to Sequence
SVD	Singular Value Decomposition
TF-IDF	Term Frequency - Inverse Document Frequency
T5	Text-to-Text Transfer Transformer
UI	User Interface

LIST OF FIGURES

Figure 1.1: Incremental Development	3
Figure 3.1: Use case diagram.....	9
Figure 3.2: Class diagram	12
Figure 3.3: Object diagram	13
Figure 3.4: State diagram.....	14
Figure 3.5: Sequence diagram.....	15
Figure 3.6: Activity diagram.....	16
Figure 4.1: Refined class diagram	17
Figure 4.2: Refined object diagram	18
Figure 4.3: Refined state diagram.....	19
Figure 4.4: Refined activity diagram	20
Figure 4.5: Component diagram	21
Figure 4.6: Deployment diagram	22
Figure 4.3: T5 Encoder-Decoder structure	27
Figure 5.1: Vocabulary IDF values	32
Figure 5.2: TF-IDF values after L2 normalization	32
Figure 5.3: Cosine similarity matrix	33
Figure 5.4: Sentence similarity	34
Figure 5.5: Page rank algorithm.....	34
Figure 5.6: Extractive summary result.....	35
Figure 5.7: Training arguments of abstractive summarization	36
Figure 5.8: Training process	37
Figure 5.9: Abstractive summary result	37
Figure 5.10: Registration module	38
Figure 5.11: Login module.....	38
Figure 5.12: User profile module.....	39
Figure 5.13: User history	39
Figure 5.14: Manage user module	40
Figure 5.15: Analytics module - 1.....	40
Figure 5.16: Analytics module - 2.....	41

LIST OF TABLES

Table 5.1: Unit testing for login module	42
Table 5.2: Unit testing for summarizer module	43
Table 5.3: System testing	44

CHAPTER 1:

INTRODUCTION

1.1 Introduction

InfoDigest is an AI-powered web application designed to combat information overload, particularly within the scientific and research communities. The platform intelligently condenses large volumes of text from various sources, including research papers, into concise and understandable summaries.

At its core, InfoDigest leverages advanced AI to provide text summarization using both extractive and abstractive approaches. It also supports document summarization for formats like .docx, .txt, and .pdf. To enhance the user experience, the system includes a simple user interface, text analysis tools (like keyword extraction), and a robust user authentication system. Logged-in users can manage their history of summaries and provide feedback, while administrators have access to user management and system analytics. InfoDigest is a powerful tool for streamlining research and knowledge management.

1.2 Problem Statement

In today's digital world, we're all dealing with a ton of information, especially in science and research. This "information overload" is a big problem. Researchers, students, and professionals have to read a lot of long, complicated documents, which takes a lot of time and energy. This slows down their work, makes it harder to review research, and can even slow down new discoveries.

The trouble is most tools for summarizing text aren't good enough. They can't handle the tricky, technical language in academic papers. Many of them are also missing key features, like different ways to summarize text, detailed analysis, and ways for users to manage their work.

So, there's a real need for a better, AI-powered website. This website would not only accurately summarize scientific papers but also offer a full set of tools to manage information more easily. It would include features that focus on the user, like a way to give feedback, and tools for admins to manage everything. The goal is to make the research process smoother and make complex information easier for everyone to access.

1.3 Objectives

The objectives of InfoDigest project are as follows:

1. To develop an AI-powered text summarization solution, using Python for intelligent model creation and deploying a MERN-based web interface for users to access and condense textual content and file input into concise summaries.
2. To utilize Artificial Intelligence and Machine Learning algorithms, specifically focusing on Natural Language Processing (NLP) methodologies for extractive and abstractive summarization, to enable InfoDigest to intelligently process and condense text.

1.4 Scope and Limitation

1.4.1. Scope

The scope of InfoDigest are as follows:

1. The system will generate summaries from various text sources, including direct text input and uploaded documents in .docx, .txt, and .pdf formats and support both extractive and abstractive summarization techniques.
2. The application will provide basic text analysis metrics, such as word count and keyword extraction, to give users a quick overview of their content.
3. A simple, clean, and easy-to-navigate interface will be a core part of the design, ensuring a smooth user experience.
4. The system will include a login and authentication system to manage user access. It will have two distinct user roles: a standard user and an administrator.
5. Logged-in users will have the ability to view a history of their past summaries.
6. The system will provide a mechanism for users to submit feedback on the quality of the generated summaries, which will be used to improve the AI models over time.
7. The admin role will have access to a dashboard to manage users and gain insights into the overall system usage. This will include analytics related to the number of summaries generated and user activity.

1.4.2. Limitations

The limitations of InfoDigest are as follows:

1. Initial development will focus solely on English language text summarization. Support for other languages is outside the current scope.

2. Advanced NLP features such as sentiment analysis, named entity, topic modeling, or question-answering are not included.
3. The project is limited to processing static textual content and cannot summarize live audio or video feeds.

1.5 Development Methodology

For InfoDigest, an incremental development approach is used. This means that instead of building the entire app at once, the system is developed in small, working parts, or "increments," with each new version adding more features to the last. This allows delivery of basic, working product early and add more functionality over time. The development process consists of the following increments:

1. The first increment will focus on the most important part: the core summarization engine, which will handle basic text input.
2. Then, the second increment will add user authentication, user history, and a simple user interface. This provides a functional app that users can start using and gives valuable feedback.
3. The third phase will expand the summarization capability to handle .docx, .txt, and .pdf files, along with adding text analysis tools and a user feedback mechanism.
4. Finally, the last increment will focus on the administrative side, building the admin dashboard for user management and system analytics.

This phased approach is ideal because it allows a working product to be delivered to users faster, incorporates their feedback at each stage, and handles any problems early on, ensuring the app truly meets their needs.

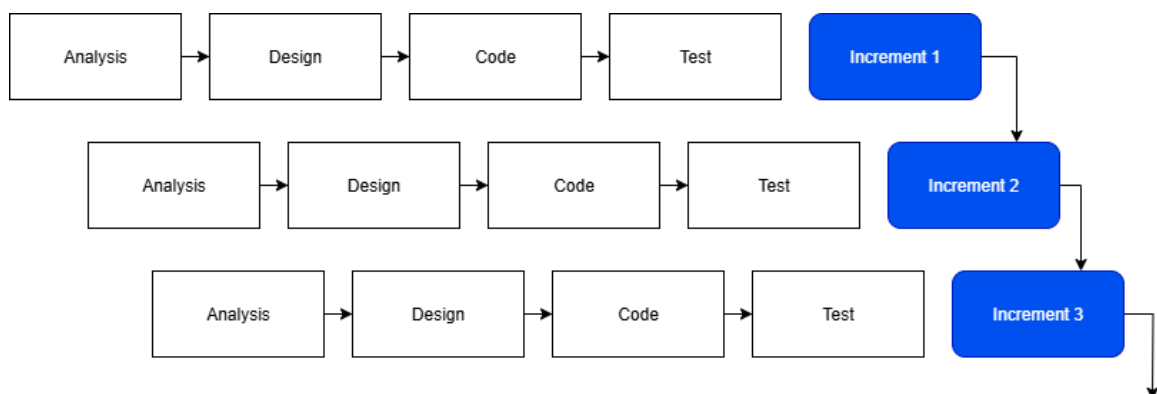


Figure 1.1: Incremental Development

1.6 Report Organization

Chapter 1: Introduction

Introduces the project, its goals, methodology, and scope.

Chapter 2: Background Study and Literature Review

Presents background theories, concepts, and previous works related to project.

Chapter 3: System Design and Architecture

Describes the overall structure of the system, including architectural components, data flow, and user role interactions.

Chapter 4: System Implementation

Provides implementation details such as the tools, programming languages, frameworks used, integration of algorithm, and includes relevant system interface screenshots.

Chapter 5: Results and Evaluation

Discusses the output of the model, evaluates its performance using accuracy and other metrics, and analyzes how well the system meets its objectives.

Chapter 6: Conclusion and Future Enhancements

Concludes the project by summarizing key achievements and outlines potential improvements and additional features for future development.

CHAPTER 2:

BACKGROUND STUDY AND LITERATURE REVIEW

2.1 Background Study

Grammarly's summarization tool, integrated within its comprehensive AI writing platform, provides users with the capability to distill lengthy documents, articles, or notes into concise overviews. This feature leverages artificial intelligence to identify and extract the most critical information, presenting it in a digestible format that aids quick comprehension and efficient information retrieval, complementing Grammarly's core grammar and style correction services [1].

QuillBot's summarizer stands as a prominent feature alongside its renowned paraphrasing capabilities, offering users a versatile tool to condense text effectively. It allows flexible summarization, enabling users to choose between extracting key sentences or generating a paragraph summary, and provides control over the output length, making it highly adaptable for various academic, professional, or personal comprehension needs [2].

Scribbr's text summarizer is specifically tailored to assist students and researchers in navigating academic literature by quickly extracting the core arguments and essential information from scholarly articles, essays, or research papers. This tool focuses on providing a rapid understanding of complex academic texts, helping users to efficiently identify main points and key findings without needing to read the entire document [3].

Summarizer.org offers a direct and user-friendly online tool dedicated solely to text summarization, providing a straightforward solution for condensing any given text. It primarily employs extractive summarization techniques, allowing users to paste content and quickly generate a shorter version, often with options to specify the desired summary length by sentence count or percentage, serving as a quick utility for basic text condensation [4].

2.2 Literature Review

Hybrid Extractive-Abstractive Summarization with Retrieval-Augmented Generation proposes a novel method that fuses extractive and abstractive approaches using a Retrieval-Augmented Generation (RAG) framework. The method first retrieves salient sentences from the source document using extractive strategies, which are then passed to a pretrained

transformer (e.g., BART or T5) to generate coherent summaries. This hybrid structure leverages the factual grounding of extractive summaries and the fluency of abstractive models. It is particularly effective in handling long-form and multi-document summarization tasks, and enhances factual accuracy, content coverage, and linguistic naturalness [5].

The paper “Attention Is All You Need” introduced the Transformer architecture, a breakthrough model for sequence-to-sequence tasks that replaced recurrence and convolution entirely with self-attention mechanisms. The Transformer uses an encoder–decoder structure, where each layer is built from multi-head self-attention, position-wise feed-forward networks, residual connections, and layer normalization. To capture word order without recurrence, the model incorporates positional encoding. This design allows for efficient parallelization, drastically reducing training time while better modeling long-range dependencies compared to RNNs and CNNs. On benchmarks like WMT 2014 machine translation (English–German, English–French), Transformers achieved state-of-the-art results, demonstrating both higher accuracy and scalability. The paper’s impact has been immense: it laid the foundation for modern large language models such as BERT, GPT, and T5, extended beyond NLP into computer vision, speech, and multimodal AI, and fundamentally shifted the research paradigm—establishing attention as the dominant framework in deep learning [6].

Automatic Text Summarization with Machine Learning: An Overview by Luis Fred offers a conceptual introduction to a range of summarization methods, categorized into extractive and abstractive approaches. The article outlines classical extractive techniques like TF-IDF, clustering, TextRank, and LSA, before transitioning into neural network-based summarization. It briefly introduces Seq2Seq models, transformers, and pretrained models such as BERT and GPT. While not academically peer-reviewed, it provides valuable insights into how machine learning and deep learning have shaped the evolution of automatic summarization [7].

Evaluation of Extractive and Abstractive Approaches for Text Summarization offers a comprehensive comparison of classical and modern summarization methods. The paper evaluates TextRank, Luhn's algorithm, and TF-IDF-based extractive methods alongside T5, PEGASUS, and BART abstractive models. The authors assess these models based on ROUGE, BLEU, and readability scores, concluding that while extractive techniques are computationally cheaper and interpretable, abstractive models outperform them in

linguistic fluency and semantic depth. The paper also suggests hybrid methods as a potential solution to combine the strengths of both paradigms [8].

Text Summarization using Deep Learning: A Study on Automatic Summarization reviews modern deep learning techniques applied to automatic summarization. The paper emphasizes encoder-decoder frameworks, including RNNs, LSTMs, and transformers, with a strong focus on models like BART, GPT, and T5. It discusses model training strategies, including fine-tuning on domain-specific datasets and the use of reinforcement learning to optimize summary quality. The study highlights the benefits of deep learning in generating coherent, context-aware, and human-like summaries while acknowledging the need for substantial computational resources [9].

Text Summarization Using Latent Semantic Analysis presents one of the earliest and most cited methods for unsupervised extractive summarization. The method constructs a term-sentence matrix from the input text and applies Singular Value Decomposition (SVD) to uncover latent semantic structures. Sentences are then selected based on their relevance to the dominant latent concepts. This ensures semantic richness and diversity in the resulting summary, even though the approach is purely statistical and not context aware. LSA remains a popular baseline due to its unsupervised nature and effectiveness in low-data environments [10].

CHAPTER 3:

SYSTEM ANALYSIS

3.1 System Analysis

3.1.1 Requirement Analysis

3.1.1.1 Functional Requirement

The functional requirements of InfoDigest are as follows:

1. Text Summarization: The system must generate summaries from various text sources like input text and uploaded files to produce summarized content. Summarization can be produced by either extractive approach or abstractive approach.
2. Document Summarization: The system must summarize text from uploaded documents (.docx, .txt, .pdf).
3. User Interface: The system provides a simple and easy to navigate interface with suitable designing.
4. Text Analysis: The system must provide text analyzer metrics (word count, key word extraction, etc.).
5. User Feedback: The system allows logged in users to send feedback on the summary generated.
6. User History: The user can view their history which displays the information related to the generated summaries.
7. User Authentication: The user needs to login to access certain features. The user can be either a normal user or an admin.
8. Admin Functionalities: The admin has the ability to manage users and obtain overall analytics of the system.

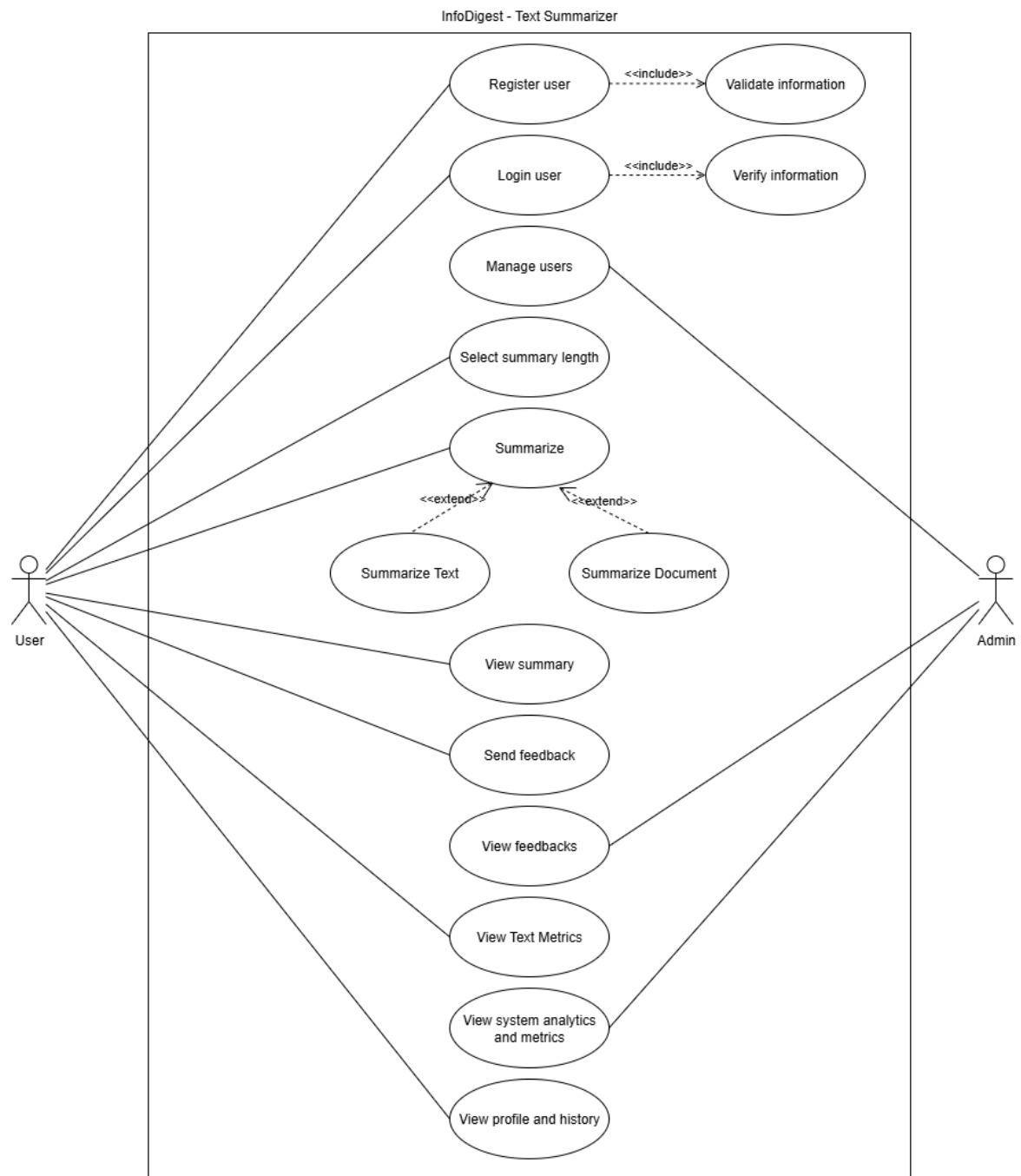


Figure 3.1: Use case diagram

This diagram illustrates how a user interacts with the InfoDigest system. The user can perform several actions: summarize text or documents by adjusting the summary length and send feedback and view text metrics. The user can also register and log in as an optional feature so that the user can store the summarization history along with their metrics. Similarly, the admin can manage users and view the user's feedback and view overall analytics and metrics of the system.

3.1.1.2 Non-Functional Requirement

The non-functional requirements of InfoDigest are as follows:

1. Performance: The system must generate summaries within an acceptable time frame and must be reasonably fast and accurate.
2. Usability: The system must have a user-friendly interface that is easy to navigate and understand.
3. Reliability: The system must be robust and handle errors gracefully.
4. Security: The system must protect user data and ensure privacy.
5. Maintainability: The system must be designed for easy maintenance and updates.

3.1.2 Feasibility Analysis

3.1.2.1 Technical feasibility

The provided text describes the technical feasibility of the InfoDigest project, highlighting its core components and architecture. The project is considered feasible because it uses a robust hybrid summarization system, combining an abstractive model (LongT5 fine-tuned with PyTorch on the arXiv dataset) with extractive methods (TF-IDF, Cosine Similarity, TextRank using NLTK). The system is deployed using a scalable FastAPI backend and a user-friendly MERN stack frontend. The use of OpenCV and PyTorch further enhances their capability, making the project a viable and well-architected solution.

Minimum requirements for technical feasibility include:

- Proficiency in Python, NLP/ML, JavaScript, React, Express.js, Node.js, FastAPI and database management (MongoDB)
- Access to a machine with sufficient computational resources for model development and training (GPU recommended for complex models)
- A development environment capable of supporting the chosen technologies and integration workflows.

3.1.2.2 Operational feasibility

InfoDigest is operationally feasible due to its intuitive, web-based interface and use of standard AI/ML and web technologies, which significantly simplify its integration into existing workflows and ultimately reduce user overhead. Furthermore, the reliance on established Artificial Intelligence, Machine Learning, and web technologies means that the underlying infrastructure benefits from robust community support, extensive

documentation, and a readily available pool of expertise for development, deployment, and ongoing maintenance. Additionally, the project's defined scope is a critical factor, as it precisely outlines what will and will not be included, preventing feature creeping and thereby minimizing the initial operational burdens associated with a more complex system.

3.1.2.3 Economic feasibility

From an academic perspective, the InfoDigest project is economically feasible because it efficiently uses the student's time and effort to achieve valuable educational outcomes. The project's potential benefits, including enhanced skills, a stronger portfolio, and possible research opportunities, outweigh the primary costs of the student's time. Careful planning, efficient resource allocation, and a focus on delivering a functional application are key to maximizing these academic gains.

Also, given that most of the technologies used in the project are freely available and cost effective, this application is financially viable.

3.1.2.4 Schedule feasibility

The InfoDigest project's schedule is feasible within a reasonable timeframe. This is supported by its defined scope, allowing focus by including feasible features. The project's modular components permit parallel development, and it will leverage existing resources to complete the project on time.

3.1.3 Analysis

InfoDigest project is developed using a object oriented approach. The object-oriented approach is a programming paradigm that structures software design around objects, rather than functions or logic. The detailed analysis of the project is displayed below:

3.1.3.1 Object modelling

Object modeling is a way to design a software system by viewing it as a collection of interacting objects. This approach focuses on defining the objects within the system, their data (attributes), and their behaviors (methods). The goal is to create a model that mirrors the real-world problem, making the system easier to understand, design, and maintain.

1. Class diagram

A Class Diagram is a core part of object modeling. It is a static, high-level blueprint that shows the system's structure. It depicts the different classes and the relationships between them, such as inheritance, association, and aggregation. Each

class is represented by a box showing its name, attributes, and methods. It provides a visual representation of the overall design and architecture of the software.

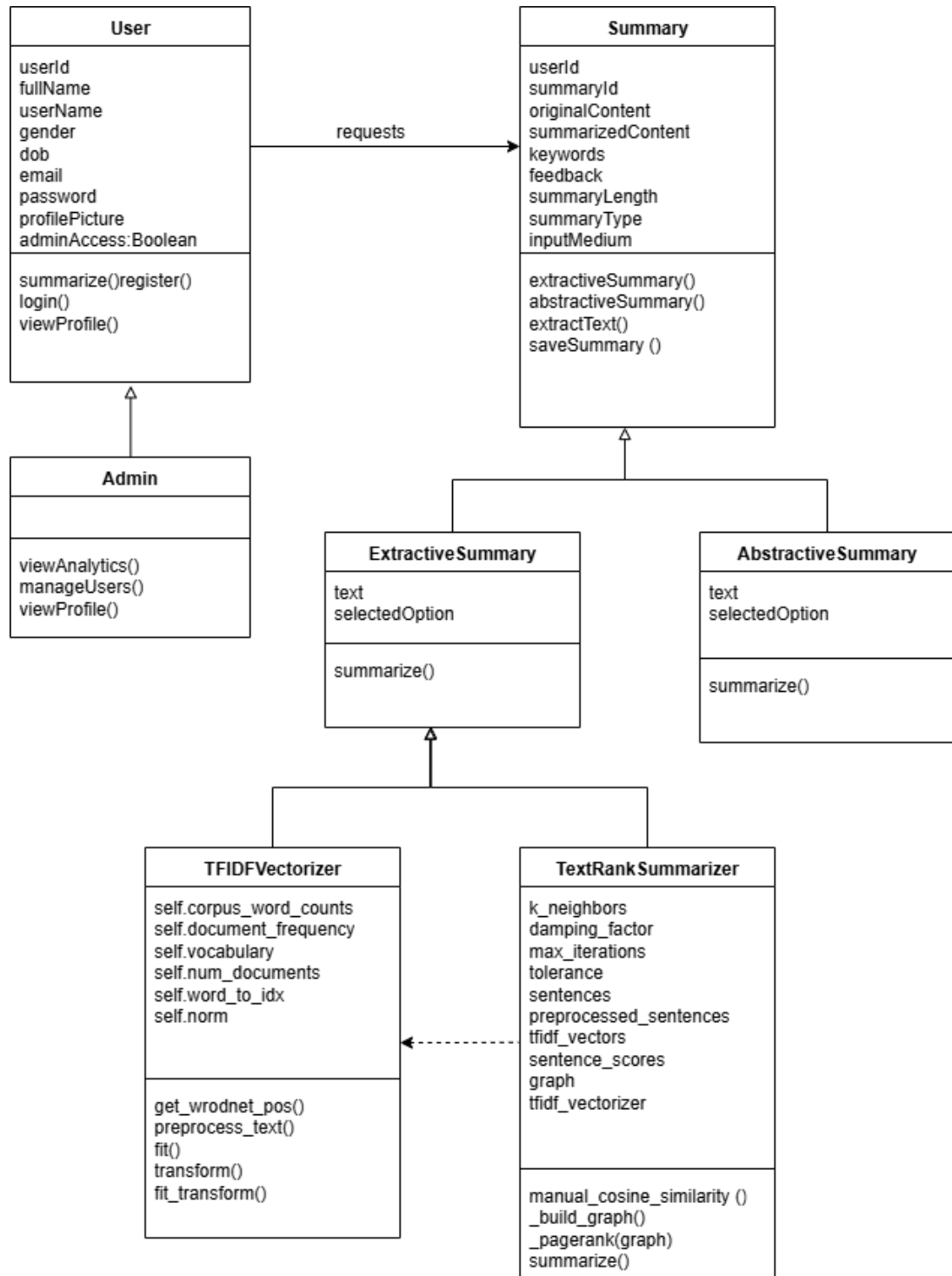


Figure 3.2: Class diagram

2. Object diagram

An Object Diagram is a specific snapshot of the system at a particular moment in time. While a Class Diagram shows the general structure, an Object Diagram shows concrete instances of those classes (the objects themselves). It displays the objects, their specific attribute values, and the links between them. This diagram is useful for demonstrating a specific scenario or for testing if the class structure can support the intended interactions.

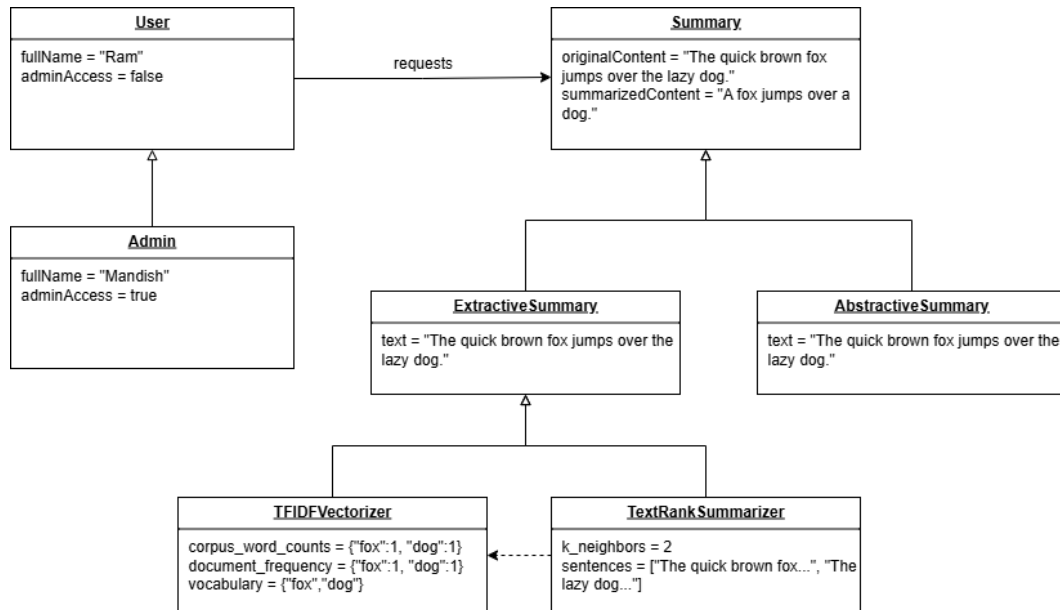


Figure 3.3: Object diagram

3.1.3.2 Dynamic modelling

Dynamic modeling describes the behavior of a system over time. It focuses on how the system's objects interact with each other and change their states in response to events.

1. State diagram

A state diagram, also known as a state machine diagram, is a behavioral diagram that shows the different states an object can have and the events that cause a transition from one state to another.

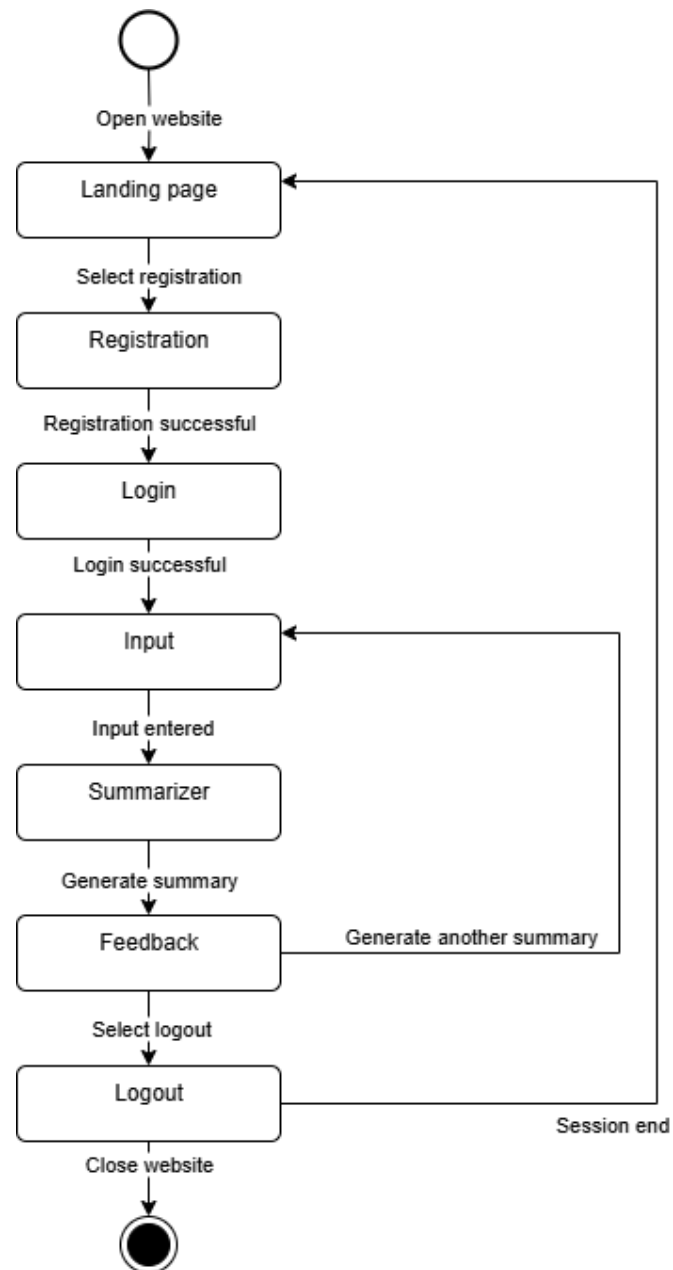


Figure 3.4: State diagram

2. Sequence diagram

A sequence diagram is a type of interaction diagram that shows how processes and objects interact with each other in a sequential manner. It depicts the chronological order of a series of messages exchanged between different objects or actors to perform a specific task.

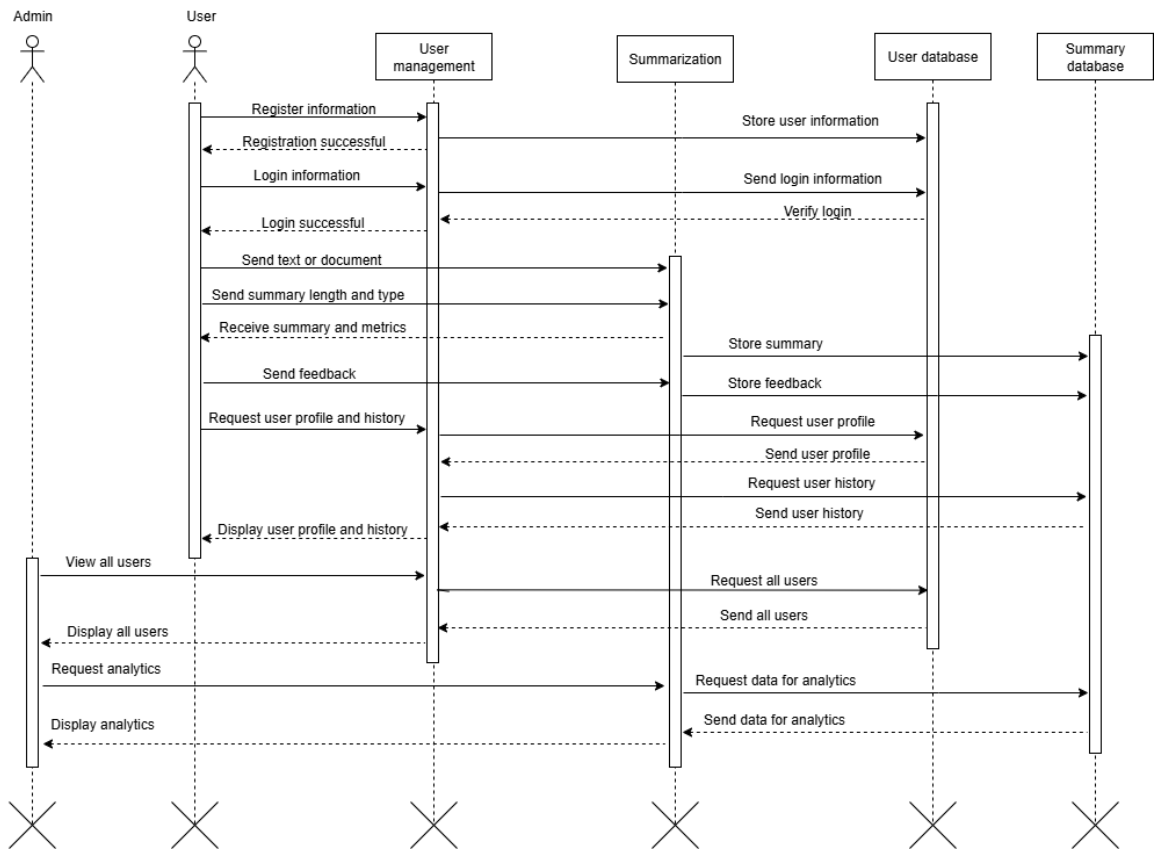


Figure 3.5: Sequence diagram

The figure demonstrates the distinct workflows for a standard user and an administrator, from initial authentication to core functionalities. The diagram shows how a user can register, log in, submit text for summarization, and access their history, while also detailing how an administrator can manage users and request system-wide analytics, all through a series of sequential messages between the user interface and various backend services like user management, summarization, and their respective databases.

3.1.3.3 Process modelling

Process modeling is the graphical representation of a business process or workflow, used to analyze and improve how work is done within an organization. It provides a common language for stakeholders to understand a sequence of activities, events, and decisions, making it easier to identify inefficiencies and opportunities for automation.

1. Activity diagram

An activity diagram is a behavioral diagram that shows the flow of control from one activity to another. It models the step-by-step workflow of a system, representing the sequence of actions, decisions, and parallel processes.

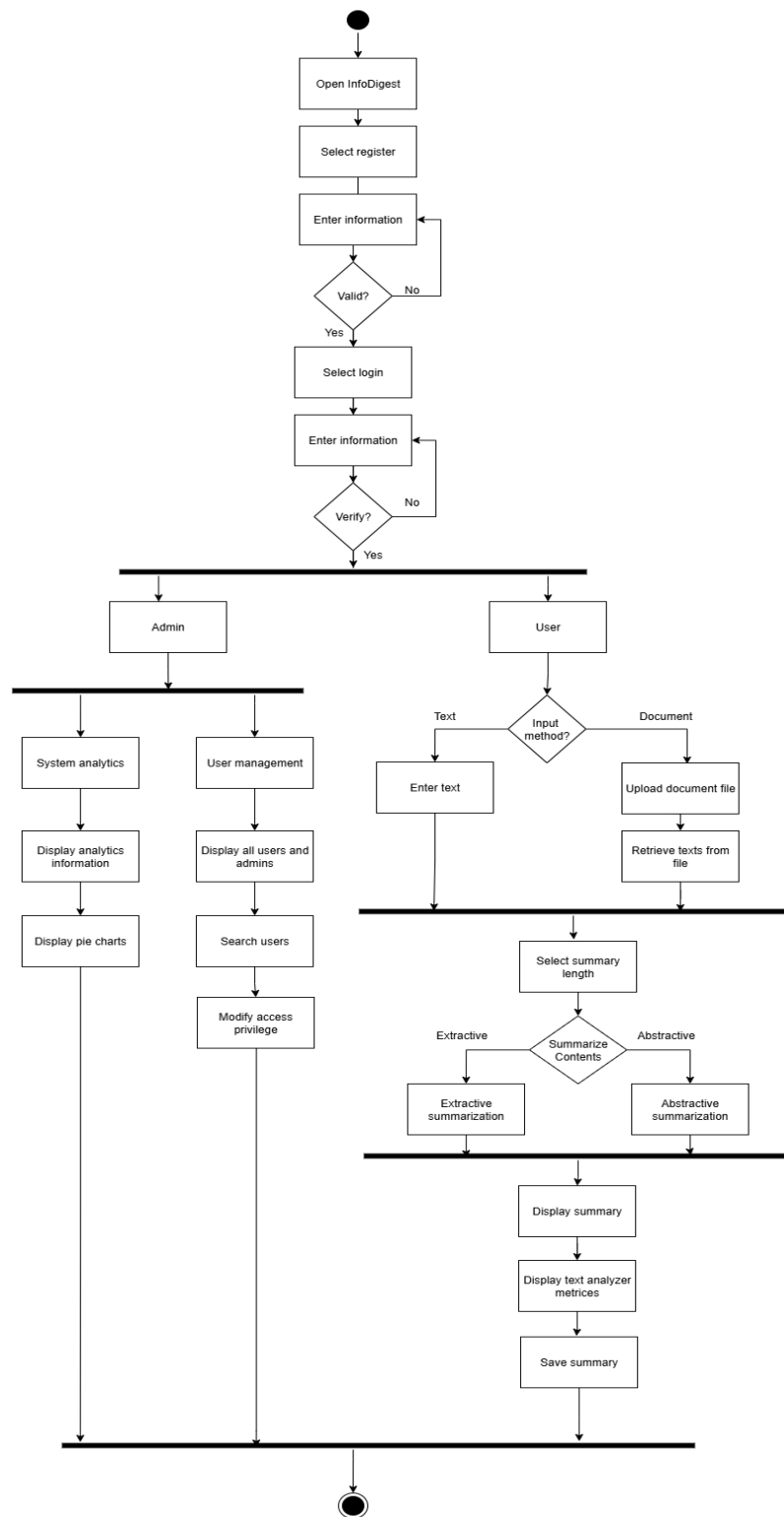


Figure 3.6: Activity diagram

CHAPTER 4:

SYSTEM DESIGN

4.1 Design

InfoDigest project is developed using object-oriented approach. It emphasizes modularity, reusability, and flexibility by representing real-world entities as objects, each containing data and methods. The detailed design of the project is displayed below:

4.1.1 Refinement of Class, Object, State, Sequence and Activity diagrams

1. Class diagram

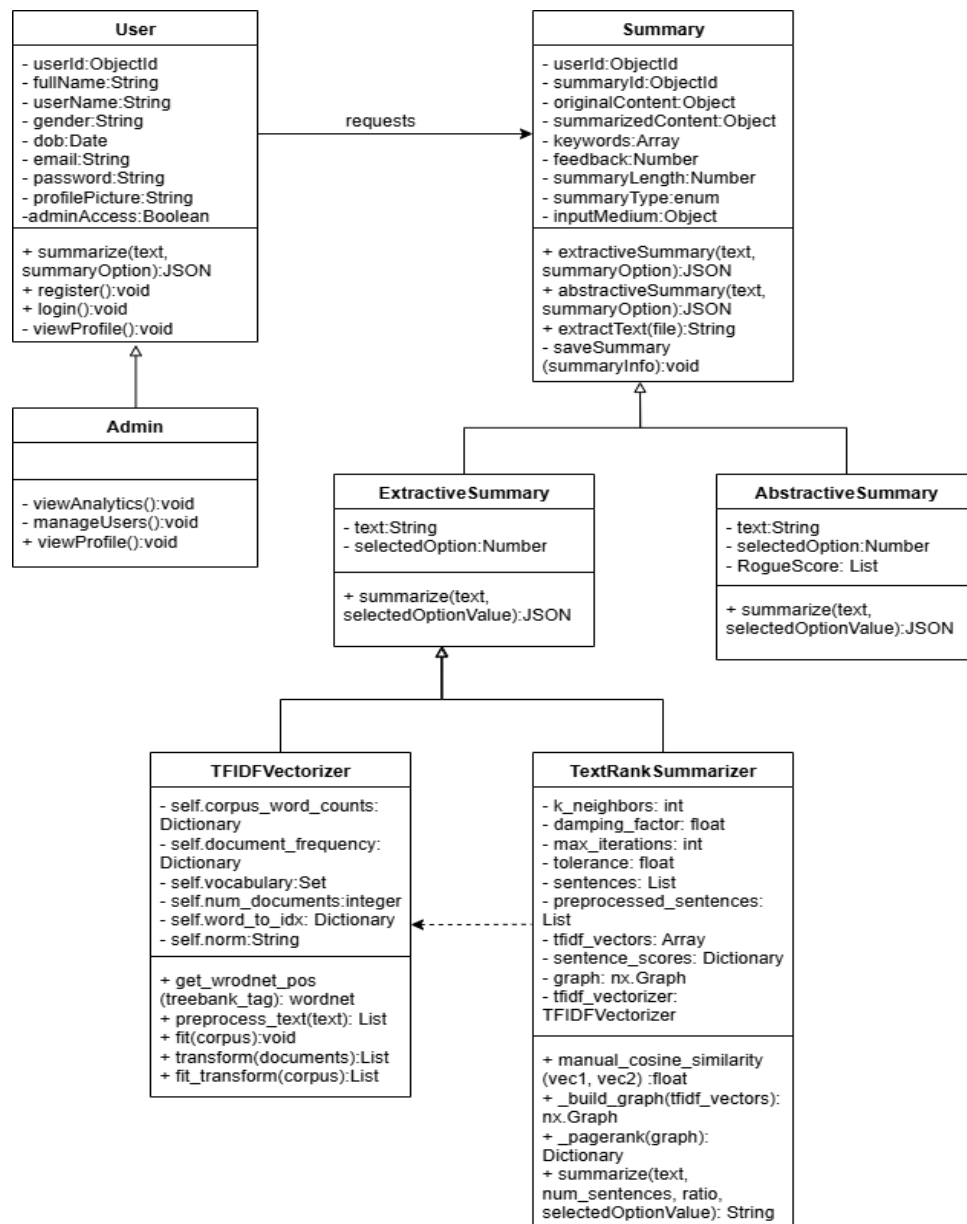


Figure 4.1: Refined class diagram

This class diagram in figure f4.1 shows a text summarization system where users request summaries and admins manage the system. The Summary class handles storing content, generating summaries, and saving results. Summarization is done through ExtractiveSummary (using TextRankSummarizer and TFIDFVectorizer) or AbstractiveSummary. The TextRankSummarizer builds graphs and applies PageRank, while TFIDFVectorizer preprocesses and vectorizes text. Overall, it illustrates how user data, summary generation, and algorithms connect to deliver summaries.

2. Object diagram

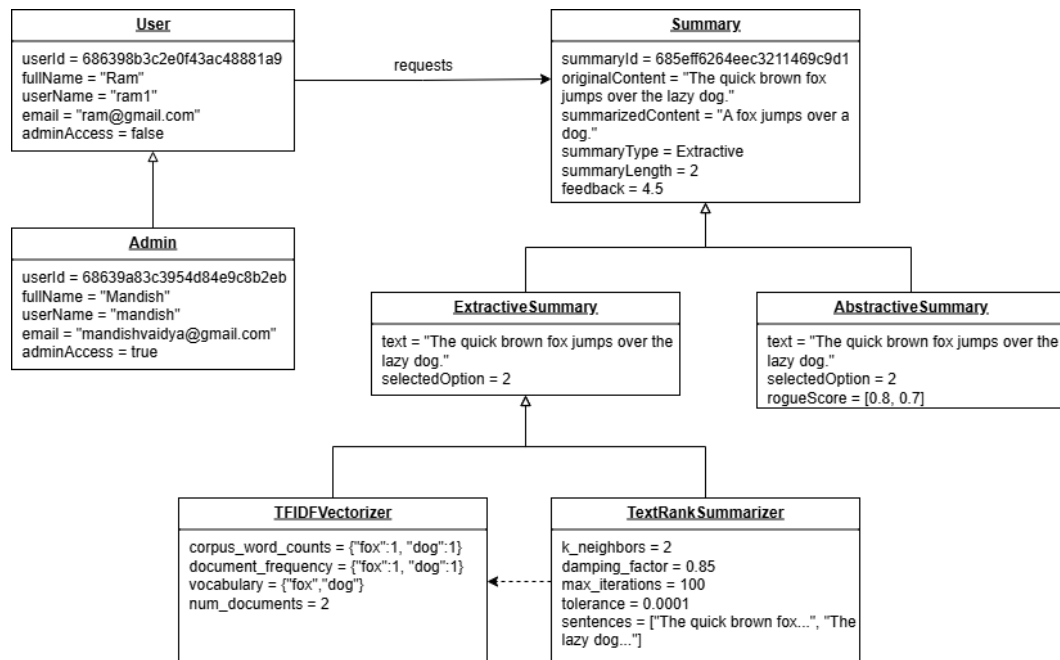


Figure 4.2: Refined object diagram

This object diagram in figure 4.2 shows a runtime snapshot of the summarization system. Here, a User (Ram) requests a summary, which is represented by a Summary object containing original content, summarized output, type, length, and feedback. An Admin (Mandish) exists with elevated access rights. The Summary is linked to an ExtractiveSummary that processes the text using the TextRankSummarizer, which in turn relies on a TFIDFVectorizer for vectorization. The diagram also includes an AbstractiveSummary object with text, option, and ROUGE score values. Overall, it demonstrates how actual objects interact at runtime to generate summaries from user input.

3. State diagram

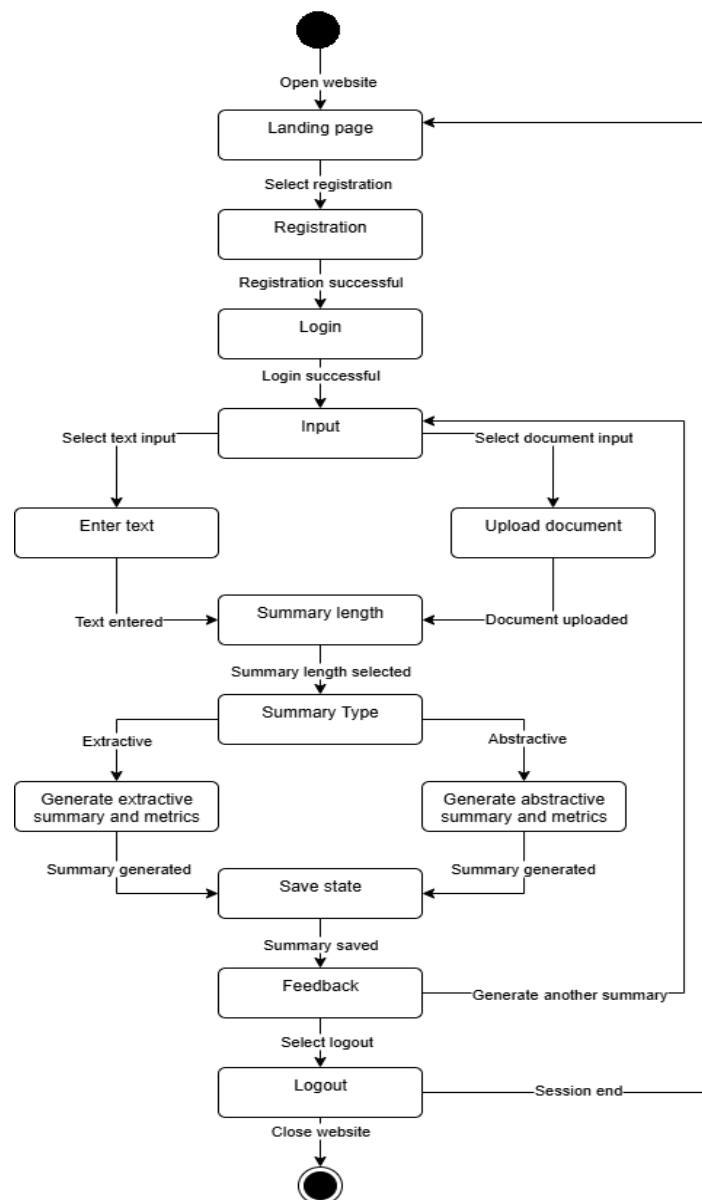


Figure 4.3: Refined state diagram

The image in figure 4.3 shows a State Diagram that outlines the user flow of InfoDigest. The process begins when a user opens the website, leading to a landing page where they can choose to either register or log in. After successful authentication, the user is directed to the input section, where they can either type in text or upload a document. The system then prompts the user to select the desired summary length and type (either extractive or abstractive). Once the summary is generated, the user can save the result, provide feedback, or choose to create another summary, which returns them to the input screen. The process concludes when the user selects to log out or ends their session, which leads to closing the website.

4. Activity diagram

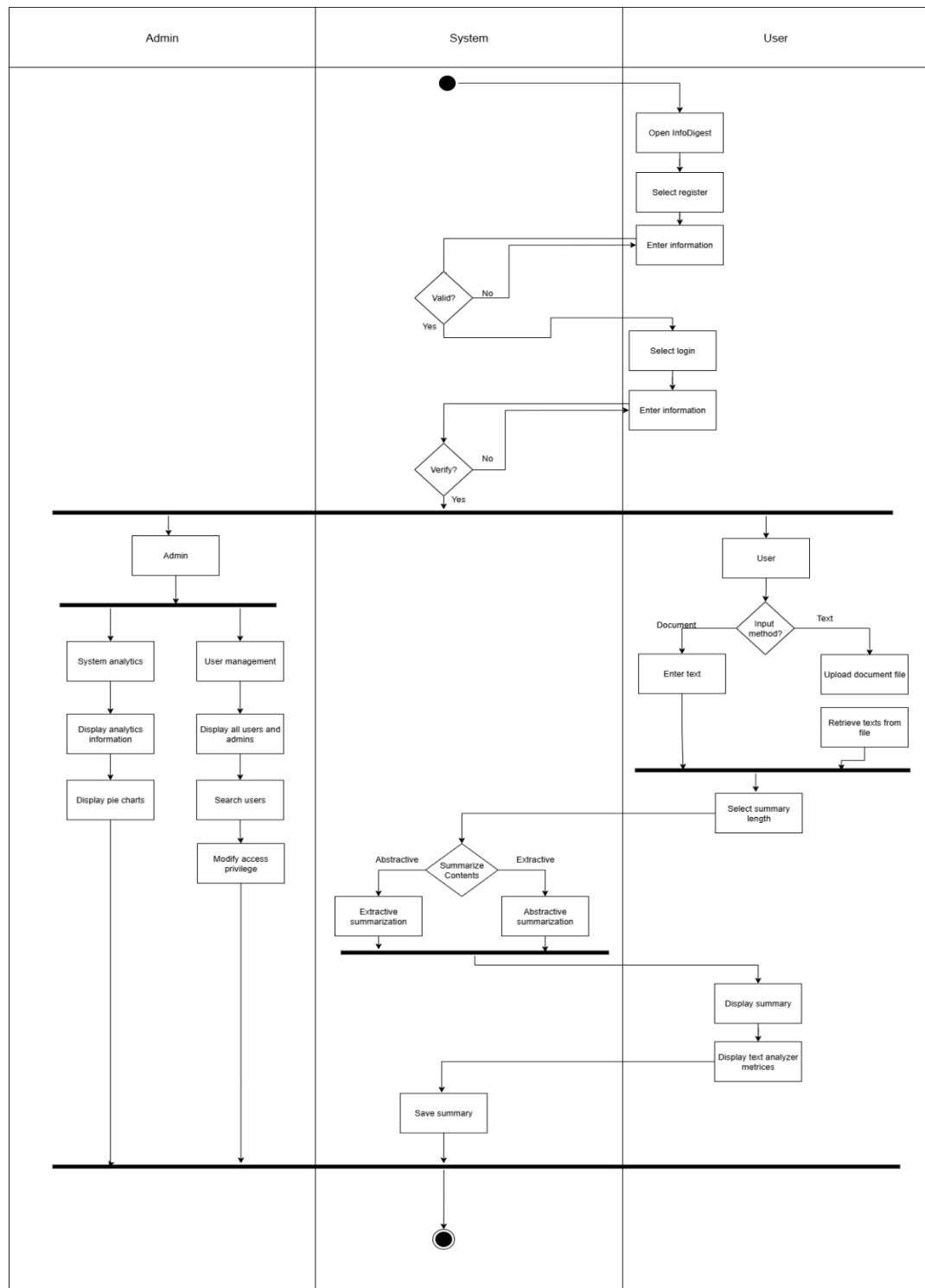


Figure 4.4: Refined activity diagram

The image in 4.4 is an activity diagram for the InfoDigest application. It begins with the user registration and login process. After successful authentication, the flow diverges based on the user type: Admin or a regular User. The User path details the process of either

inputting text or uploading a document, selecting a summarization method (extractive or abstractive), and then viewing and saving the generated summary. The admin path outlines separate functionalities for managing user accounts and viewing system analytics, which are displayed in the form of metrics or pie charts. This diagram visually maps out the entire operational workflow of the application.

4.1.2 Component diagrams

A component diagram is used to illustrate the high-level architecture of a system, showing how the major components are organized and how they depend on each other. It provides a static view of the system's modular structure.

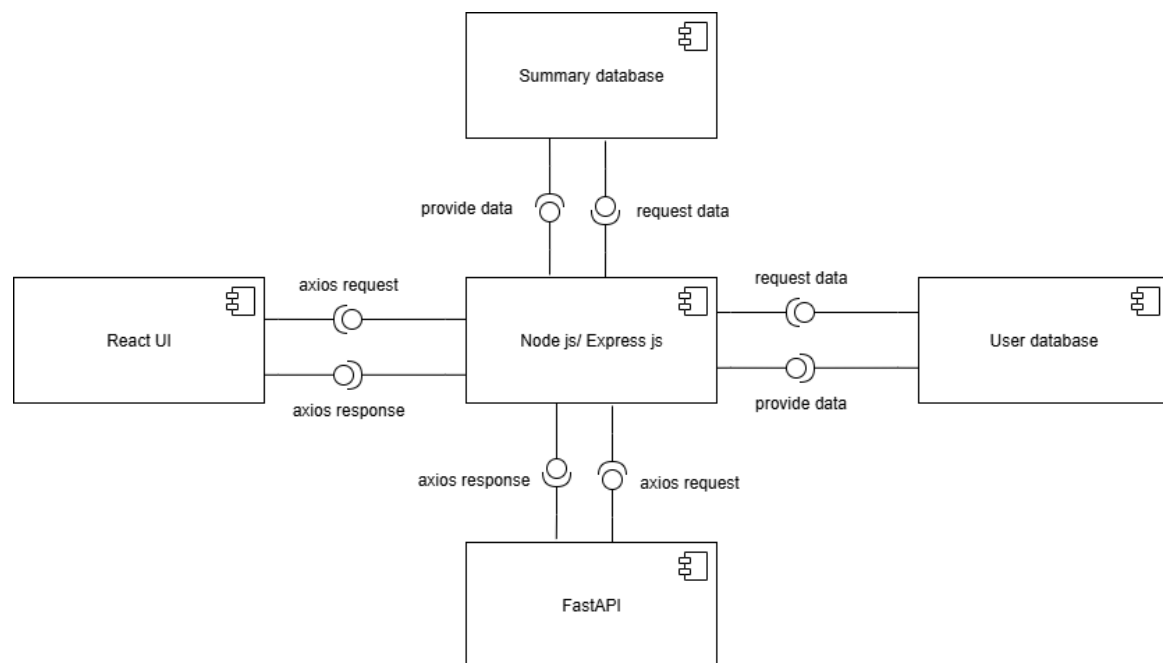


Figure 4.5: Component diagram

The image in 4.5 represents a component diagram for InfoDigest. At its center is the Node.js/Express.js backend, acting as the main hub that communicates with all other components. The React UI frontend sends and receives requests and responses from the backend using Axios. The FastAPI service, which handles complex summarization and analysis tasks, also communicates with the central backend via Axios. Finally, the two separate databases, User database and Summary database, are shown, with the main backend requesting and providing data to each to manage user information and store generated summaries respectively.

4.1.3 Deployment diagrams

A deployment diagram illustrates the physical architecture of a system, showing how the different software components are deployed across various hardware nodes and the connections between them. It provides a view of the physical environment where the system will run.

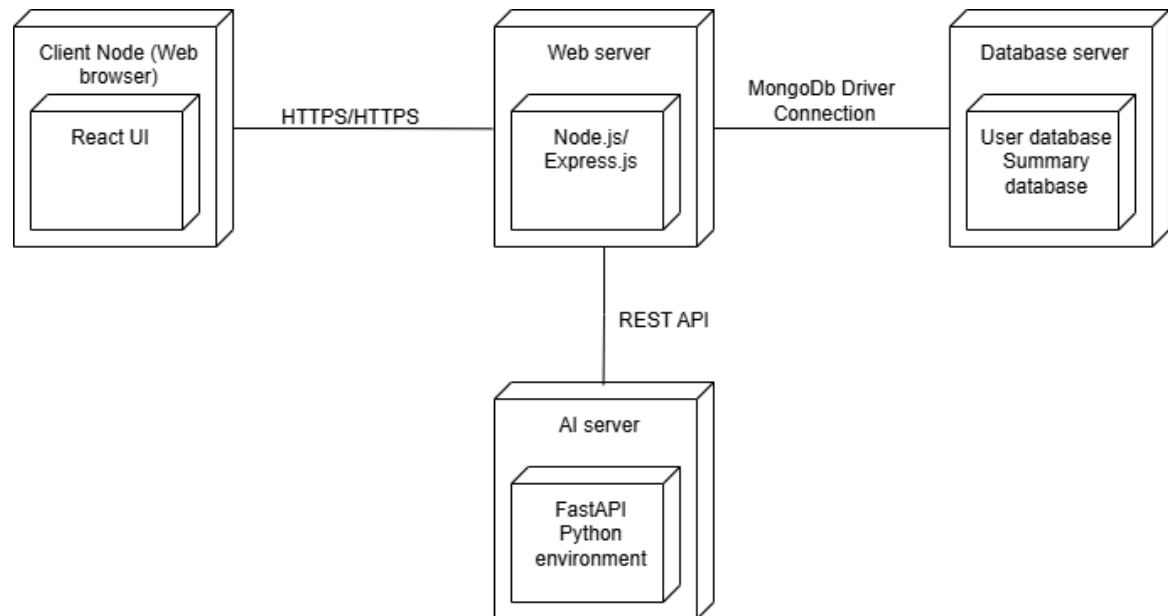


Figure 4.6: Deployment diagram

The figure in 4.6 is a deployment diagram for InfoDigest. It shows a client node representing the user's browser, where the React UI is deployed. This client communicates via HTTPS with a Web server, which hosts the Node.js/Express.js backend. This central backend then communicates with an AI server that hosts the FastAPI service via a REST API. Additionally, the backend connects to a Database server to interact with both the User database and Summary database via a MongoDB Driver Connection.

4.2 Algorithm Details

The algorithm details for the InfoDigest project are as follows:

4.2.1 Extractive Summarization

Extractive summarization is a technique where a summary is generated by selecting significant sentences or phrases directly from the original document. Unlike abstractive summarization, it does not generate new text but rather extracts existing content. This

approach relies on identifying the most important parts of a text based on various linguistic and statistical features.

1. Term Frequency-Inverse Document Frequency (TF-IDF)

TF-IDF is a numerical statistic that reflects how important a word is to a document in a collection or corpus. It is often used as a weighting factor in information retrieval and text mining.

The Term Frequency (TF) measures how frequently a term appears in a document:

$$TF(t, d) = \frac{\text{Number of times term } t \text{ appears in document } d}{\text{Total number of terms in document } d}$$

The Inverse Document Frequency (IDF) measures how important a term is:

$$IDF(t, D) = \log \left(\frac{\text{Total number of documents } N}{\text{Number of documents with term } t \text{ in them}} \right)$$

Finally, TF-IDF is calculated as the product of TF and IDF:

$$TF-IDF(t, d, D) = TF(t, d) \times IDF(t, D)$$

Higher TF-IDF scores indicate higher importance of a word in a specific document relative to the entire corpus. This metric can be used to assign scores to words within sentences, which then contribute to the overall importance score of a sentence.

2. Cosine Similarity

Cosine similarity is a measure of similarity between two non-zero vectors of an inner product space. It measures the cosine of the angle between them. If the vectors are normalized to unit length, the dot product is the cosine similarity. For text summarization, sentences are often represented as vectors (e.g., using TF-IDF weights or word embeddings), and cosine similarity is used to find how similar two sentences are.

The formula for cosine similarity between two vectors A and B is:

$$similarity(A, B) = \frac{A \cdot B}{||A|| \cdot ||B||} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}}$$

Where A_i and B_i are components of vector A and B respectively. A value of 1 means the vectors are identical, 0 means they are orthogonal (no similarity), and -1 means they are diametrically opposed. In summarization, this helps identify redundant sentences or group similar sentences together.

3. K-Nearest Neighbors (K-Neighbors) for Graph Construction

While k-neighbors is typically a classification algorithm, its concept can be adapted to build a graph for text summarization algorithms like TextRank. In this context, each sentence in the document is a node in the graph. The edges between sentences are determined by their similarity (e.g., using cosine similarity).

To build the graph using a k-neighbors approach for TextRank:

- Each sentence is converted into a numerical vector using TF-IDF weights of its words.
- The cosine similarity between every pair of sentence vectors is calculated.
- For each sentence (node), identify its k most similar sentences based on the cosine similarity scores. An edge is drawn between the current sentence node and its k nearest neighbors. The weight of this edge can be the cosine similarity score itself. The value of k can be determined dynamically based on the total number of sentences in the input text:
 - for texts with up to 15 sentences, k is 2
 - for texts between 16 and 100 sentences, k is 5
 - for texts with over 100 sentences, k is 10.

This creates a weighted graph where nodes are sentences and edges represent their similarity. The correct value of k is not fixed and should be altered as per the desired output.

This graph structure is crucial for algorithms like TextRank, as it models the relationships and interconnections between sentences in the document.

4. TextRank Algorithm

TextRank is a graph-based ranking algorithm for text processing, inspired by Google's PageRank algorithm. It identifies the most important sentences in a text based on their "connectivity" within the graph of sentences.

The steps for TextRank are:

- The input text is divided into sentences, and each sentence is tokenized and preprocessed (e.g., stop word removal, stemming/lemmatization, part-of-speech tagging).
- Each sentence is represented as a node in a graph $G=(V,E)$, where V is the set of vertices (sentences) and E is the set of edges between them.

- Edges are drawn between two sentences if their similarity score (e.g., cosine similarity based on shared words or TF-IDF vectors) exceeds a certain threshold, or by using a k-neighbors approach as described above. The edges are weighted by their similarity scores.
- The algorithm iteratively calculates a score for each sentence, similar to how PageRank scores web pages. The score of a sentence depends on the scores of the sentences it is connected to, and the weights of those connections. The TextRank score $S(V_i)$ for a vertex (sentence) V_i is given by:

$$S(V_i) = (1 - d) + d \times \sum_{V_j \in \text{In}(V_i)} \frac{w_{ji}}{\sum_{V_k \in \text{Out}(V_j)} w_{jk}} S(V_j)$$

Where:

- d is the damping factor (typically set to 0.85), which represents the probability of jumping to a random vertex.
- $\text{In}(V_i)$ is the set of vertices that point to V_i (sentences similar to V_i).
- $\text{Out}(V_j)$ is the set of vertices that V_j points to (sentences that V_j is similar to).
- w_{ji} is the weight of the edge from V_j to V_i (e.g., their cosine similarity).

This process is iterated until the scores converge.

Once the scores for all sentences are stable, the sentences are ranked in descending order of their TextRank scores. The top-ranked sentences, up to a desired length or word count, are then selected to form the extractive summary, usually maintaining their original order within the document for coherence.

4.2.2 Abstractive algorithm

An abstractive summarization algorithm generates a summary by creating new sentences and phrases that capture the main ideas of the source text, rather than simply extracting existing sentences. This process requires a deep understanding of the source material.

The LongT5 transformer model is a type of abstractive summarization algorithm designed to handle very long documents, a common limitation for many transformer models. It is built upon the T5 (Text-to-Text Transfer Transformer) architecture but incorporates a local-global attention mechanism. This allows the model to process long sequences of text

efficiently without a significant performance drop. Essentially, it can pay attention to all parts of a long document, enabling it to synthesize information from various sections to create a cohesive summary.

By using either local or transient-global attention, the model can scale to process input sequences of up to 16,384 tokens, a major leap from the typical 512-token limit of many transformer models.

1. T5 framework

The T5 (Text-to-Text Transfer Transformer) framework is a unified model that handles a wide range of NLP tasks by treating every problem as a **text-to-text** task. This means that for any given input, the model's output is always a string of text. This approach simplifies the architecture and allows the same model, with the same training process, to be used for different tasks like summarization, question answering, translation, and classification.

- Translation: “translate English to German: How are you?” → “Wie geht es dir?”
- Summarization: “summarize: The article discusses climate change impacts...” → “Climate change affects ecosystems.”
- Classification: “sst2 sentence: The movie was great. label:” → “positive”

2. Encoder–Decoder Structure of LongT5

The LongT5 model is an extension of the T5 framework, and it retains the core encoder-decoder structure while making a crucial modification to handle extremely long sequences of text.

The T5 framework treats text summarization as a text-to-text problem, using a standard encoder-decoder transformer architecture. This unified approach simplifies the process, as the same model and code can be applied to many different NLP tasks simply by changing the input format.

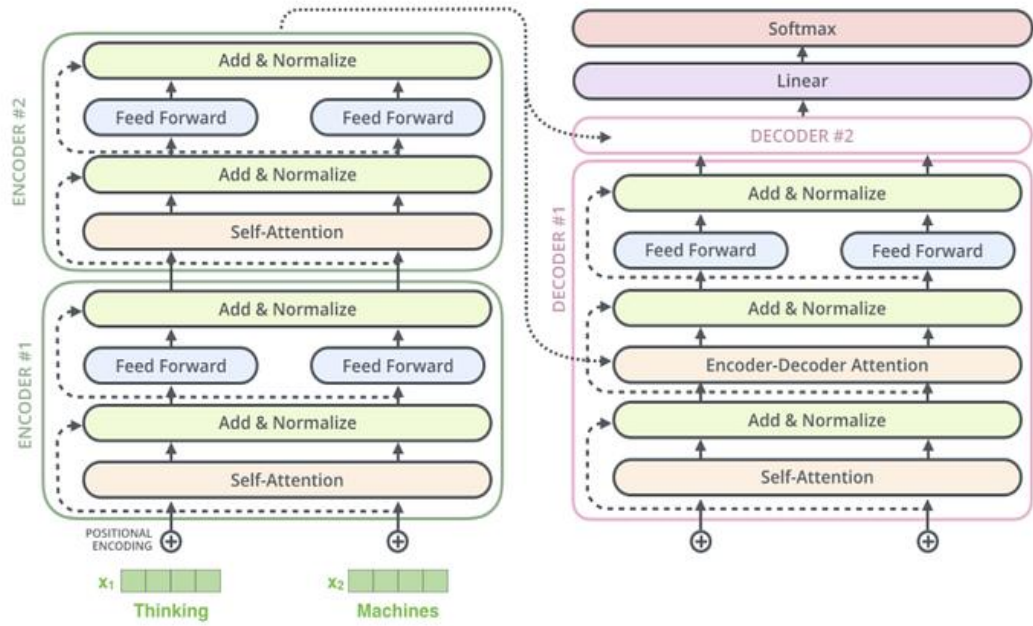


Figure 4.7: T5 Encoder-Decoder structure

The LongT5 model consists of two main parts:

- **Encoder:** The encoder's job is to read the entire input document and create a compressed, contextual representation. In standard T5, this is done with a full self-attention mechanism, which has a quadratic computational cost. LongT5 addresses this by replacing the standard attention with a more efficient, hybrid mechanism that scales linearly with input length. This mechanism uses a combination of two techniques:
 - **Local Attention:** Each token only attends to a fixed window of tokens around it (e.g., 128 tokens to the left and 128 to the right). This is computationally efficient and captures the local context well. This approach restricts attention to a fixed window of tokens around each token (e.g., 127 tokens to the left and right). The complexity is linear with respect to the input sequence length (l) and the window size (r), or $O(l \times r)$. Since r is a constant, this scales as $O(l)$, a major efficiency improvement over the quadratic $O(l^2)$ of standard attention.
 - **Transient-Global Attention:** To prevent the model from losing the "big picture," certain key tokens are designated as "global" tokens. These global tokens attend to all tokens in the sequence, and all other tokens attend to them. This creates a bottleneck that forces the model to distill global context into a

few representations, which the decoder can then access. The complexity of this mechanism is $O(l(r+1/k))$, where k is the block size used to create the global tokens. This also offers a significant efficiency gain over standard attention for long sequences.

This hybrid approach allows the LongT5 encoder to process long documents without the memory and computational burden of full attention.

- **Decoder:** The decoder's structure is largely similar to the standard T5 decoder. The decoder generates the summary based on the encoder's representation. It uses masked self-attention to generate the summary word by word and cross-attention to link its output to the encoder's understanding of the source document. This streamlined process allows LongT5 to deliver state-of-the-art results on long-form summarization tasks while being computationally practical. It uses two types of attention:
 - **Decoder Self-Attention:** This is a masked attention mechanism that ensures the decoder can only "see" the tokens it has already generated. This allows for sequential text generation.
 - **Encoder-Decoder Cross-Attention:** This is the crucial link between the encoder and decoder. The decoder uses this mechanism to "query" the encoder's output, essentially asking "Which parts of the original document are most relevant to what I'm generating right now?". This allows the decoder to produce a summary that is semantically aligned with the source text, even as it creates new phrases and sentences. The LongT5 decoder can use a standard cross-attention mechanism because the output summary is typically much shorter than the input document, so the quadratic scaling issue is not a concern here.

The decoder hidden states are projected to the vocabulary with a softmax layer. The probability of a summary sequence $y = (y_1, \dots, y_m)$ given the input document x is:

$$P(y|x) = \prod_{t=1}^m P(y_t|y_{<t}, x)$$

This ensures that the summary is both fluent (depending on past tokens) and faithful (depending on input).

CHAPTER 5:

IMPLEMENTATION AND TESTING

5.1 Implementation

5.1.1. Tools Used

The different tools and software used in the project are as follows:

- **Platform:** Windows
- **Front end:** React
- **Back end:** Express.js, FastAPI
- **Database:** MongoDB
- **IDE:** VS Code
- **API Testing:** Postman
- **Model creation:** Python, Jupyter Notebook
- **Dataset:** ccdv/arxiv-summarization, CNN/DailyMail
- **Environment Management:** Anaconda
- **GPU:** NVIDIA GeForce RTX 3060
- **Diagramming tool:** Draw io

5.1.2. Implementation Details of Modules

The modules of InfoDigest are as follows:

Module 1: Extractive Summarization

The extractive summarization is used to select the highest valued sentence from among the original corpus. The code for extractive summarization is written in python so, it is integrated in the MERN project using FastAPI to create API of the python functions.

```

@app.post("/api/extractive-summary")
async def api_extractive_summary(request: ExtractiveSummarizerRequest):
    if not request.text.strip():
        raise HTTPException(status_code=400, detail="Text is required(FastAPI)")

    try:
        result = Extractive_Summarizer(request.text, request.ratio)
        original_sentences = sent_tokenize(request.text)
        original_length_sentences = len(original_sentences)
        summary_sentences = sent_tokenize(result)
        summary_length_sentences = len(summary_sentences)
        return {
            "summary": result,
            "original_length_sentences": original_length_sentences,
            "summary_sentences_count": summary_length_sentences
        }
    except Exception as e:
        raise HTTPException(status_code=500, detail=f"Error in summarizing: {str(e)}")

```

The FastAPI backend of “/api/extractive-summary” receives text and ratio as input which is used to perform extractive summary. It consists of the following imports:

```

import string
import math
from collections import Counter
import numpy as np
import pandas as pd
import networkx as nx
import matplotlib.pyplot as plt
import nltk
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
from nltk.corpus import wordnet
from nltk.tokenize import sent_tokenize

```

The Extractive_Summarizer consists of 2 main classes:

i. **TFIDFVectorizer**

This function is used to calculate the TF-IDF vectors of the given input. The key components for a TF-IDF calculation system. self.corpus_word_counts will store the frequency of each word within individual documents, while self.document_frequency tracks how many different documents contain a particular word. self.vocabulary will hold all unique words encountered across the entire collection of documents, and self.num_documents will count the total number of documents processed. self.word_to_idx

creates a numerical mapping for each unique word, and `self.norm` likely indicates the type of normalization to be applied to the calculated TF-IDF scores.

```
class TfidfVectorizer:
    def __init__(self, norm='l2'):
        self.corpus_word_counts = {} # Stores word counts per document (for TF)
        self.document_frequency = {} # Stores how many documents each word appears in (for IDF)
        self.vocabulary = set()      # All unique words across the corpus
        self.num_documents = 0
        self.word_to_idx = {}        # Mapping of word to its index in the vocabulary
        self.norm = norm

    def get_wordnet_pos(self, treebank_tag):...

    def preprocess_text(self, input_text):...

    # Learns the vocabulary and document frequencies from the given corpus.
    def fit(self, corpus):...

    def _calculate_idf(self, word):...

    # Transforms a list of documents into TF-IDF vectors with optional L2 normalization.
    def transform(self, documents):...

    # Fits the vectorizer to the corpus and then transforms the corpus into TF-IDF vectors.
    def fit_transform(self, corpus):...
```

Here, the `get_wordnet_pos` function maps part-of-speech tags to WordNet's format, crucial for accurate lemmatization. The `preprocess_text` method cleans text by converting to lowercase, removing punctuation and stopwords, and performing lemmatization (reducing words to their base form) for consistent word representation.

The `fit` method is responsible for "learning" from a given corpus of documents. It iterates through each document, preprocesses it, and populates `self.vocabulary` (all unique words), `self.document_frequency` (how many documents each word appears in), and `self.corpus_word_counts` (word counts per document). It also establishes a `word_to_idx` mapping for vector representation.

The `_calculate_idf` method computes the Inverse Document Frequency for a given word using a "smooth IDF" variant, which helps prevent division by zero for words not present in the training corpus.

Finally, the `transform` method converts new documents (or the original corpus) into numerical TF-IDF vectors. For each document, it calculates the Term Frequency (TF) of each word (how often a word appears in that document) and combines it with the pre-calculated IDF to produce the TF-IDF score. Optionally, it applies L2 normalization to

these vectors, which can be useful for downstream machine learning tasks. The `fit_transform` method conveniently combines the fit and transform steps.

```
--- TF-IDF Vectorizer Vocabulary and IDF values from Summarizer ---
Vocabulary (sorted): ['1', '138', '1957', '1961', '2', '2019', 'aboard',

IDF values:
1: 4.4500
138: 4.4500
1957: 4.4500
1961: 4.4500
2: 4.4500
2019: 4.4500
aboard: 4.4500
accelerate: 4.4500
accessible: 4.4500
across: 4.4500
advance: 4.4500
age: 4.4500
ago: 4.4500
agricultural: 4.4500
alone: 4.4500
also: 4.0445
ancient: 4.0445
apollo: 4.4500
armstrong: 4.4500
around: 4.4500
```

Figure 5.1: Vocabulary IDF values

```
TF-IDF Vectors for each sentence (L2 Normalized):
Sentence 1:
endlessly : 0.4606
fascinate : 0.4606
mysterious: 0.4606
universe  : 0.3889
vast      : 0.4606
L2 Norm: 1.0000
-----
Sentence 2:
century   : 0.4238
human     : 0.3192
look      : 0.4238
night     : 0.4238
sky       : 0.4238
wonder    : 0.4238
L2 Norm: 1.0000
-----
Sentence 3:
ancient   : 0.3626
civilization: 0.3626
constellation: 0.3990
map       : 0.3990
star      : 0.3168
-----
```

Figure 5.2: TF-IDF values after L2 normalization

ii. TextRankSummarizer

The `TextRankSummarizer` class is used to enforce the TextRank algorithm to identify the most valued sentence among the corpus. The `TextRankSummarizer`'s constructor sets up the core parameters for its summarization process, including `k_neighbors` to define graph connectivity, `damping_factor` and `max_iterations` for the PageRank algorithm's behavior,

and tolerance for convergence. It also initializes internal data structures to store sentences, their processed forms, TF-IDF representations, calculated PageRank scores, and the similarity graph, along with an instance of `TFIDFVectorizer` for text numericalization.

```
class TextRankSummarizer:
    def __init__(self, k_neighbors, damping_factor=0.85, max_iterations=100, tolerance=1e-4):
        self.k_neighbors = k_neighbors
        self.damping_factor = damping_factor
        self.max_iterations = max_iterations
        self.tolerance = tolerance

        self.sentences = []
        self.preprocessed_sentences = []
        self.tfidf_vectors = None
        self.sentence_scores = None
        self.graph = None
        self.tfidf_vectorizer = TFIDFVectorizer(norm='l2')

    def manual_cosine_similarity(self, vec1, vec2): ...

    def _build_graph(self, tfidf_vectors): ...

    def _pagerank(self, graph): ...

    def summarize(self, text, num_sentences=None, ratio=None): ...
```

The `manual_cosine_similarity` function calculates the cosine similarity, a common metric in NLP, between two input vectors. It computes the dot product of the vectors and divides by the product of their magnitudes, returning 0.0 if either vector is a zero vector to prevent division by zero errors, thus providing a numerical measure of their semantic resemblance.

```
-----
--- Full Cosine Similarity Matrix ---
      Sentence 1 Sentence 2 Sentence 3 Sentence 4 Sentence 5 \
Sentence 1      1.0      0.0      0.0000      0.0000      0.0000
Sentence 2      0.0      1.0      0.0000      0.0000      0.0000
Sentence 3      0.0      0.0      1.0000      0.1169      0.0000
Sentence 4      0.0      0.0      0.1169      1.0000      0.0000
Sentence 5      0.0      0.0      0.0000      0.0000      1.0000
...
Sentence 58     0.0      0.0      0.0000      0.0000      0.0000
Sentence 59     0.0      0.0      0.0000      0.0000      0.0703
Sentence 60     0.0      0.0      0.0000      0.0000      0.0000
Sentence 61     0.0      0.0      0.0000      0.0000      0.0000
Sentence 62     0.0      0.0      0.0000      0.0000      0.0000

      Sentence 6 Sentence 7 Sentence 8 Sentence 9 Sentence 10 ...
Sentence 1      0.0      0.0      0.0000      0.0000      0.0 ...
Sentence 2      0.0      0.0      0.0000      0.0000      0.0 ...
Sentence 3      0.0      0.0      0.0000      0.0000      0.0 ...
Sentence 4      0.0      0.0      0.0000      0.0000      0.0 ...
Sentence 5      0.0      0.0      0.0000      0.0000      0.0 ...
```

Figure 5.3: Cosine similarity matrix

The private helper function `_build_graph` is crucial for constructing the sentence similarity graph, the core of the TextRank algorithm. It first computes a pairwise cosine similarity

matrix between all sentence TF-IDF vectors. Then, based on the `k_neighbors` parameter, it identifies and connects the top-K most similar sentences with weighted edges in a NetworkX graph, visually enhanced with debugging prints and Matplotlib visualizations.

```

--- Graph Edges (Relationships) with Relative Thresholding (K=5 neighbors) ---
Sentence 1 <-> Sentence 32 | Similarity: 0.1433
Sentence 1 <-> Sentence 34 | Similarity: 0.1592
Sentence 10 <-> Sentence 17 | Similarity: 0.1258
Sentence 10 <-> Sentence 24 | Similarity: 0.2161
Sentence 10 <-> Sentence 50 | Similarity: 0.1634
Sentence 11 <-> Sentence 20 | Similarity: 0.1697
Sentence 11 <-> Sentence 24 | Similarity: 0.1344
Sentence 11 <-> Sentence 29 | Similarity: 0.1431
Sentence 11 <-> Sentence 53 | Similarity: 0.1067
Sentence 12 <-> Sentence 31 | Similarity: 0.0993
Sentence 12 <-> Sentence 32 | Similarity: 0.1148
Sentence 12 <-> Sentence 46 | Similarity: 0.1092
Sentence 12 <-> Sentence 47 | Similarity: 0.0996
Sentence 12 <-> Sentence 49 | Similarity: 0.1109

```

Figure 5.4: Sentence similarity

The private helper function `_pagerank` implements a customized PageRank algorithm, central to determining sentence importance. It iteratively calculates a "score" for each sentence (node) in the graph, where scores propagate based on neighbor importance and edge weights, driven by a `damping_factor`. The process continues until scores converge or a maximum iteration limit is reached, ultimately identifying the most central and relevant sentences.

```

--- Sentence PageRank Scores ---
Sentence 23 (Score: 0.0178): Scientists believe Mars may once have supported life.
Sentence 46 (Score: 0.0178): Their research supports future missions to the Moon a
Sentence 3 (Score: 0.0174): Ancient civilizations mapped the stars and told storie
Sentence 8 (Score: 0.0174): As technology advanced, so did our reach into space...
Sentence 16 (Score: 0.0173): Since then, space exploration has only accelerated...
Sentence 41 (Score: 0.0173): Private companies are making space more accessible...
Sentence 26 (Score: 0.0171): Meanwhile, telescopes like Hubble have revealed breat
Sentence 36 (Score: 0.0170): Thousands have been discovered orbiting distant stars
Sentence 24 (Score: 0.0170): We dream of sending humans there someday....
Sentence 19 (Score: 0.0169): They carry messages from Earth for any who might find
Sentence 21 (Score: 0.0169): Rovers like Spirit, Opportunity, Curiosity, and Perse
Sentence 5 (Score: 0.0169): The invention of the telescope revolutionized our unde
Sentence 53 (Score: 0.0168): Yet, human ingenuity continues to surprise us....
Sentence 47 (Score: 0.0167): NASA plans to return to the Moon under the Artemis pr
Sentence 49 (Score: 0.0167): The Moon will serve as a testbed for deeper space mis
Sentence 29 (Score: 0.0167): In 2019, scientists captured the first image of a bla
Sentence 34 (Score: 0.0167): They make up most of the universe, yet we barely unde
Sentence 42 (Score: 0.0166): Satellites now provide internet and Earth observation

```

Figure 5.5: Page rank algorithm

The `summarize` function is the main orchestrator for text summarization. It tokenizes the input text into sentences, converts them into TF-IDF vectors using the internal

tfidf_vectorizer, and then constructs the sentence similarity graph via `_build_graph`. Subsequently, it applies `_pagerank` to score sentence importance. Finally, it sorts sentences by their scores and extracts the top-ranked ones to form the summary, either by a specified number of sentences or a ratio of the original text length.

```
--- Extractive Summary (Top 5 Sentences) ---
Ancient civilizations mapped the stars and told stories about constellations. As technology advanced, so did our reach into space. Since then, space exploration has only accelerated. Scientists believe Mars may once have supported life. Their research supports future missions to the Moon and Mars.
```

Figure 5.6: Extractive summary result

Module 2: Abstractive summarization

The abstractive summarization module consists of a robust process for fine-tuning a LongT5 model to perform abstractive summarization. The first key step involves the intelligent management of model checkpoints. The code is designed to automatically detect and load the most recent checkpoint from the results directory, which is crucial for resuming training from a previous state without starting over. This allows for long-running training sessions to be robust against interruptions. The notebook then loads the `ccdv/arxiv-summarization` dataset, which contains academic papers and their abstracts, and a custom `preprocess_function` prepares the data for the model by tokenizing and formatting the text. The articles and abstracts are truncated to specific lengths to fit the model's input requirements.

Sample:

Article: additive models @xcite provide an important family of models for semiparametric regression or classification . some reasons for the success of additive models are their increased flexibility when compared to linear or generalized linear models and their increased interpretability when compared to fully nonparametric models .

it is well - known that good estimators in additive models are in general less prone to the curse of high dimensionality than good estimators in fully nonparametric models...

Summary: additive models play an important role in semiparametric statistics .

this paper gives learning rates for regularized kernel based methods for additive models .

these learning rates compare favourably in particular in high dimensions to recent results on optimal learning rates for purely nonparametric regularized kernel based quantile regression using the gaussian radial basis function kernel , provided the assumption of an additive model is valid...

Following the data preparation, the notebook sets up the training environment. It loads the LongT5ForConditionalGeneration model and its corresponding tokenizer from the identified checkpoint. The TrainingArguments class from the transformers library is used to define key parameters for the training loop, such as the output directory, evaluation and saving strategies, learning rate, batch sizes, and the number of training epochs.

```
training_args = TrainingArguments(  
    output_dir="./results",  
    evaluation_strategy="no",  
    # eval_steps=5000,  
    save_strategy="steps",  
    save_steps=1000,  
    save_total_limit=2,  
    load_best_model_at_end=False,  
    metric_for_best_model="rougeL",  
    greater_is_better=True,  
    learning_rate=1e-5,  
    per_device_train_batch_size=1,  
    per_device_eval_batch_size=1,  
    eval_accumulation_steps=4,  
    num_train_epochs=2,  
    weight_decay=0.01,  
    logging_dir="./logs",  
    logging_steps=200,  
    fp16=False,  
    bf16=False,  
    gradient_accumulation_steps=2,  
    max_grad_norm=1.0,  
    disable_tqdm=False,  
    report_to=[],  
    save_safetensors=True,  
)
```

Figure 5.7: Training arguments of abstractive summarization

Notably, a custom callback, ClipNanGradientsCallback, is implemented to prevent gradient explosion, a common issue in deep learning that can destabilize training. This callback ensures that the gradients are clipped to a maximum norm of 1.0, promoting stable and reliable training. The Trainer object is then instantiated with the model, training arguments, datasets, and a data collator, which handles padding and batching of the tokenized data.

```
{'loss': 1.5613, 'grad_norm': 1.0089662075042725, 'learning_rate': 8.383862706039447e-08, 'epoch': 1.98}
99%|██████████| 181400/182732 [18:51<1:03:22, 2.85s/it]
{'loss': 1.6676, 'grad_norm': 1.443673849105835, 'learning_rate': 7.289363658253618e-08, 'epoch': 1.99}
99%|██████████| 181600/182732 [28:21<53:15, 2.82s/it]
{'loss': 1.6064, 'grad_norm': 1.3262107372283936, 'learning_rate': 6.194864610467789e-08, 'epoch': 1.99}
99%|██████████| 181800/182732 [37:52<44:20, 2.85s/it]
{'loss': 1.6021, 'grad_norm': 1.496598482131958, 'learning_rate': 5.100365562681961e-08, 'epoch': 1.99}
100%|██████████| 182000/182732 [47:23<34:48, 2.85s/it]
{'loss': 1.6008, 'grad_norm': 1.7560162544250488, 'learning_rate': 4.005866514896132e-08, 'epoch': 1.99}
c:\Users\PC\anaconda3\envs\longt5\lib\site-packages\transformers\modeling_utils.py:1006: FutureWarning: The `device` argument is deprecated
warnings.warn(
100%|██████████| 182200/182732 [57:22<25:35, 2.89s/it]
{'loss': 1.6195, 'grad_norm': 1.6497410535812378, 'learning_rate': 2.911367467110304e-08, 'epoch': 1.99}
100%|██████████| 182400/182732 [1:06:55<15:46, 2.85s/it]
{'loss': 1.6291, 'grad_norm': 1.2874150276184082, 'learning_rate': 1.8168684193244754e-08, 'epoch': 2.0}
100%|██████████| 182600/182732 [1:16:27<06:16, 2.85s/it]
{'loss': 1.6305, 'grad_norm': 1.576964259147644, 'learning_rate': 7.223693715386468e-09, 'epoch': 2.0}
100%|██████████| 182732/182732 [1:22:42<00:00, 36.82it/s]
{'train_runtime': 4962.859, 'train_samples_per_second': 73.64, 'train_steps_per_second': 36.82, 'train_loss': 0.015285461631534245,
```

Figure 5.8: Training process

The result of fine tuning LongT5 model after 2 epochs on arxiv dataset is as follows:

```
TrainOutput(global_step=182732, training_loss=0.015285461631534245,
metrics={'train_runtime': 4962.859, 'train_samples_per_second': 73.64,
'train_steps_per_second': 36.82, 'total_flos': 2.002116063264768e+18, 'train_loss':
0.015285461631534245, 'epoch': 1.999994527534709})
```

Finally, the notebook includes sections for generating and evaluating summaries. A dedicated function is provided to compute ROUGE scores, which are a standard metric for evaluating summarization models. The `rouge_scorer` from the `rouge_score` library is used to compare the generated summaries against the human-written reference abstracts. This quantitative evaluation, along with a qualitative example of a generated summary, provides a clear measure of the model's performance. The entire notebook forms a complete and self-contained workflow, demonstrating best practices for fine-tuning a large language model on a specific task.

```
Generated Summary:
sunspot data from 1923 - 1933 (cycle 16) are analysed. a new method of the diagnosis of an echo-effect in the power spectrum is p

ROUGE Scores:
rouge1: 0.4516
rouge2: 0.1333
rougeL: 0.3226
```

Figure 5.9: Abstractive summary result

Module 3: Registration

This module allows the user to register to the system so that they can have a personalized feel and have access to their user history. This module consists of entering user information, which the system validates so that there is no conflict of information based on username and email address.

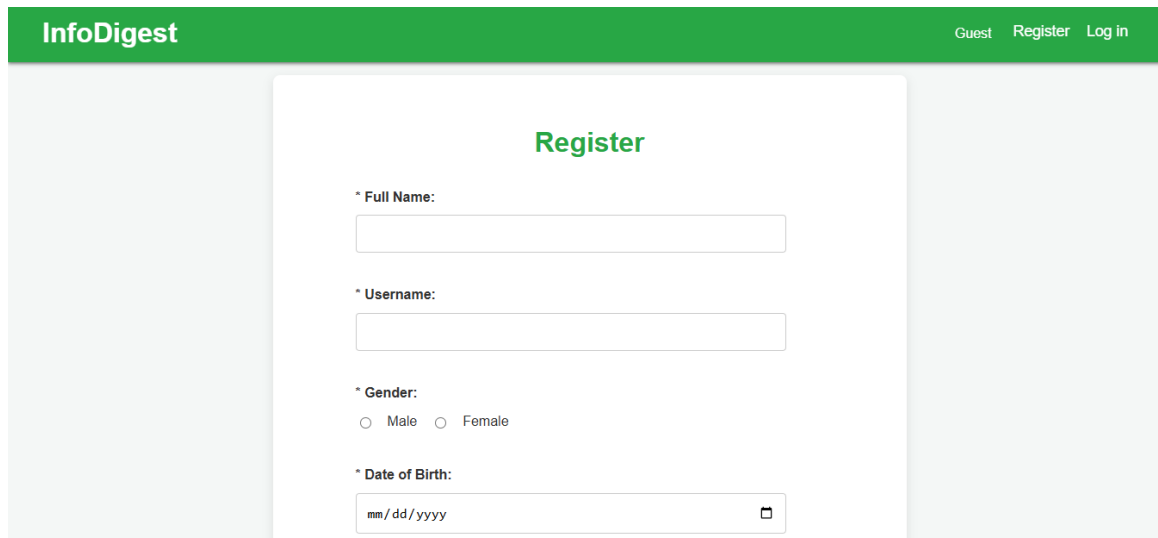
The screenshot shows the 'Register' page of the 'InfoDigest' application. The header is green with 'InfoDigest' on the left and 'Guest Register Log in' on the right. The main content area is white and centered, featuring the title 'Register' in green. Below the title are four form fields: 'Full Name:' (text input), 'Username:' (text input), 'Gender:' (radio buttons for 'Male' and 'Female'), and 'Date of Birth:' (text input with a date picker icon). The page is flanked by light gray vertical bars.

Figure 5.10: Registration module

Module 4: Login

This module allows the user to log in to the system to have access to their personal history and save activity in the system. This module consists of entering user information and password, which is verified by the system and authenticates the user.

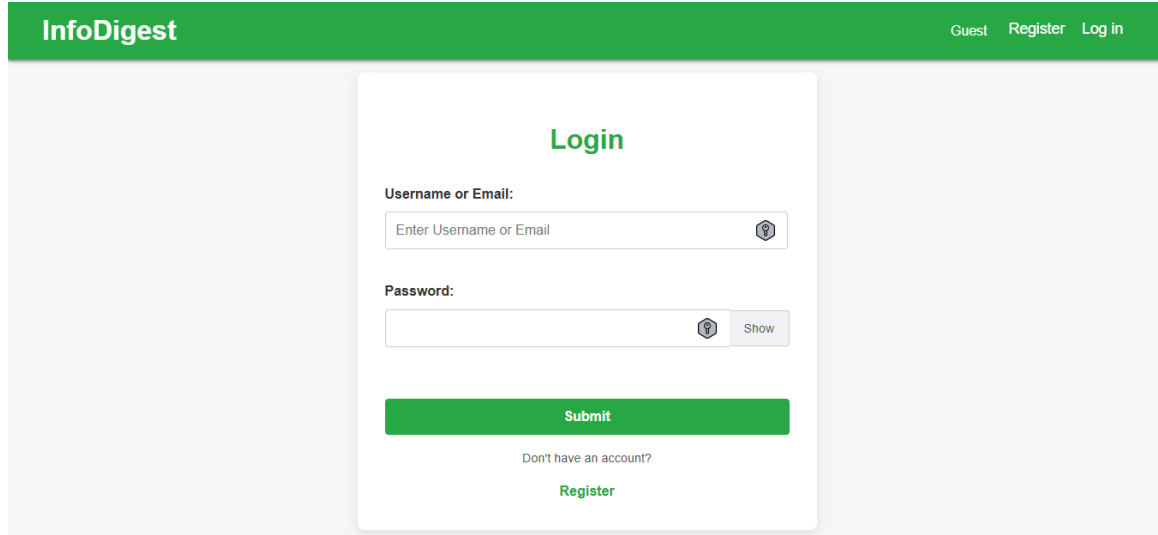
The screenshot shows the 'Login' page of the 'InfoDigest' application. The header is green with 'InfoDigest' on the left and 'Guest Register Log in' on the right. The main content area is white and centered, featuring the title 'Login' in green. Below the title are two form fields: 'Username or Email:' (text input with a placeholder 'Enter Username or Email' and a shield icon) and 'Password:' (text input with a placeholder, a shield icon, and a 'Show' button). Below these fields is a green 'Submit' button. At the bottom, there is a link 'Don't have an account?' followed by a green 'Register' link. The page is flanked by light gray vertical bars.

Figure 5.11: Login module

Module 5: User profile

This module allows the user to access their profile which consists of their user information and their history of all the summary actions performed. A user can only access user profile of themselves. An admin can access user profile of any user in the system.

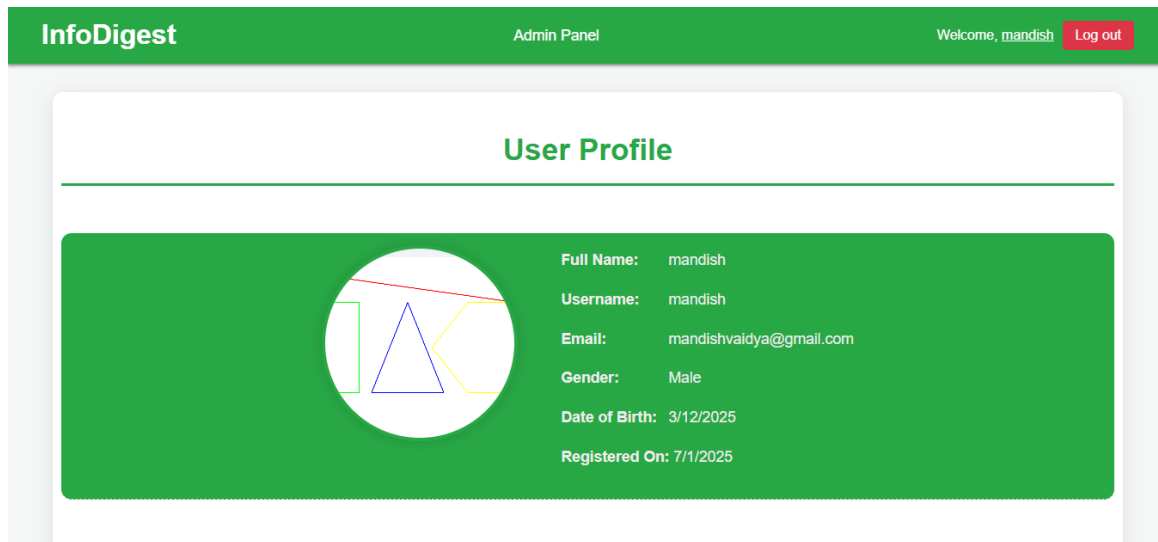


Figure 5.12: User profile module

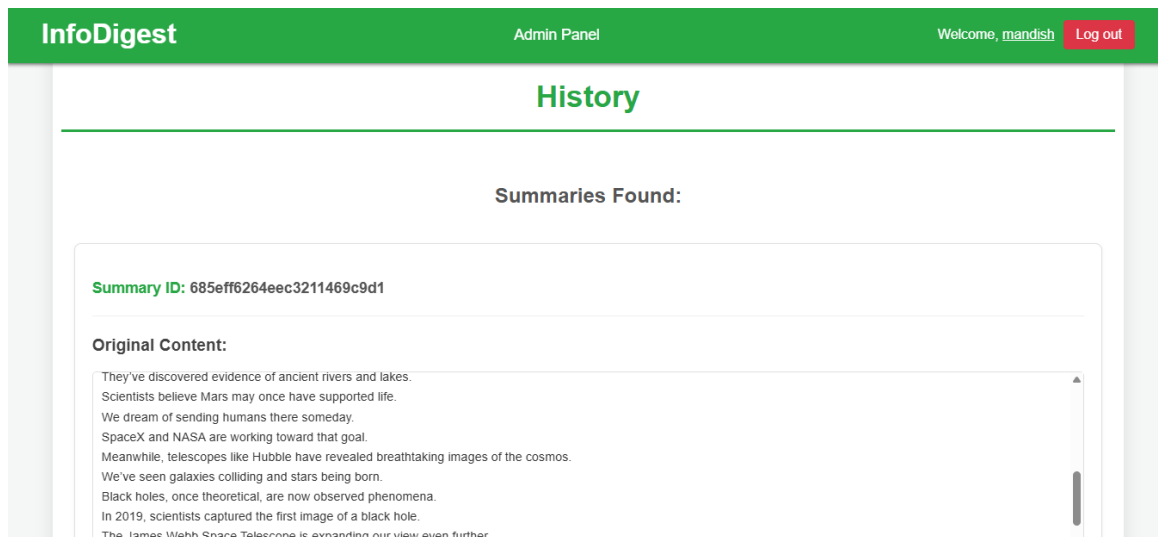


Figure 5.13: User history

Module 6: Manage users

This module is unique to only the admin of the system. This module allows the admin to view all the existing users and existing admins of the system. It also allows the admin to modify access privileges which can be set to normal user or admin. The current admin cannot modify access privileges of themselves.

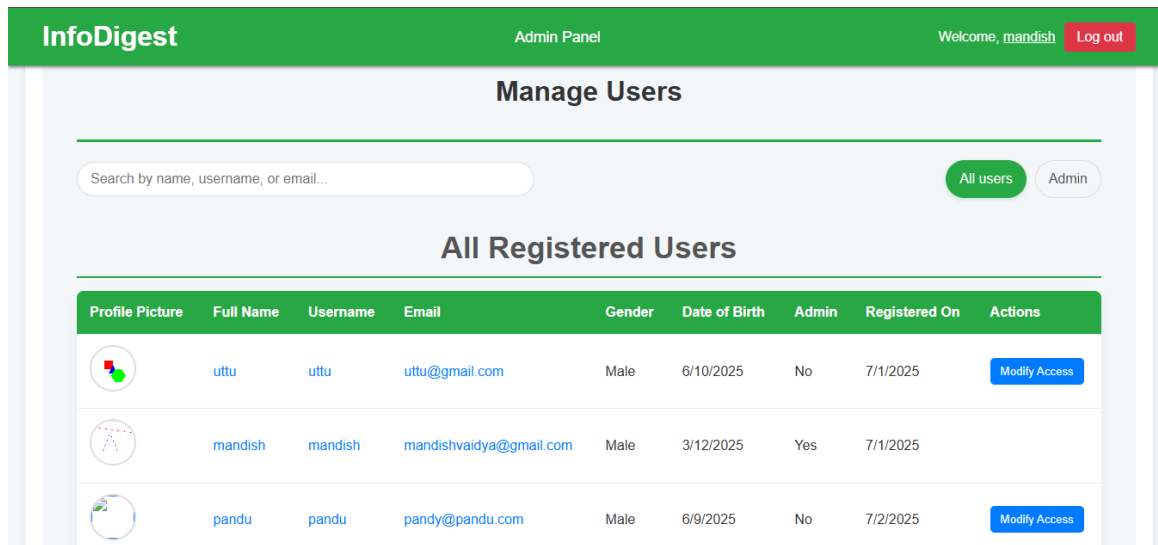


Figure 5.14: Manage user module

Module 7: Analytics

This module is unique to only the admin of the system. This module allows the admin to view all the analytics of the system. It displays statistical information along with pie charts to visualize and summarize the information related to the system and its activities.

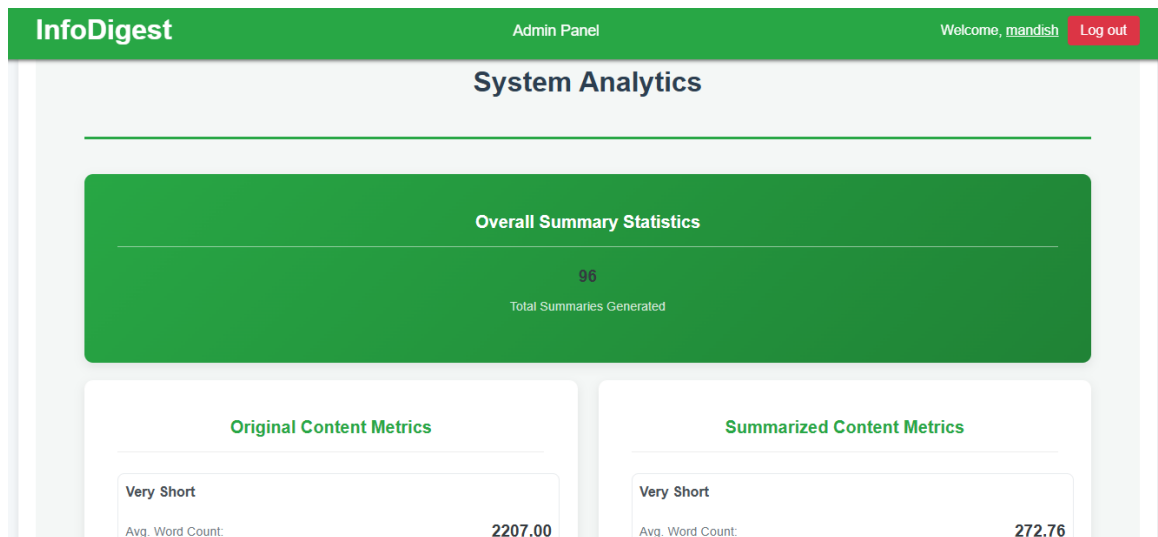


Figure 5.15: Analytics module - 1

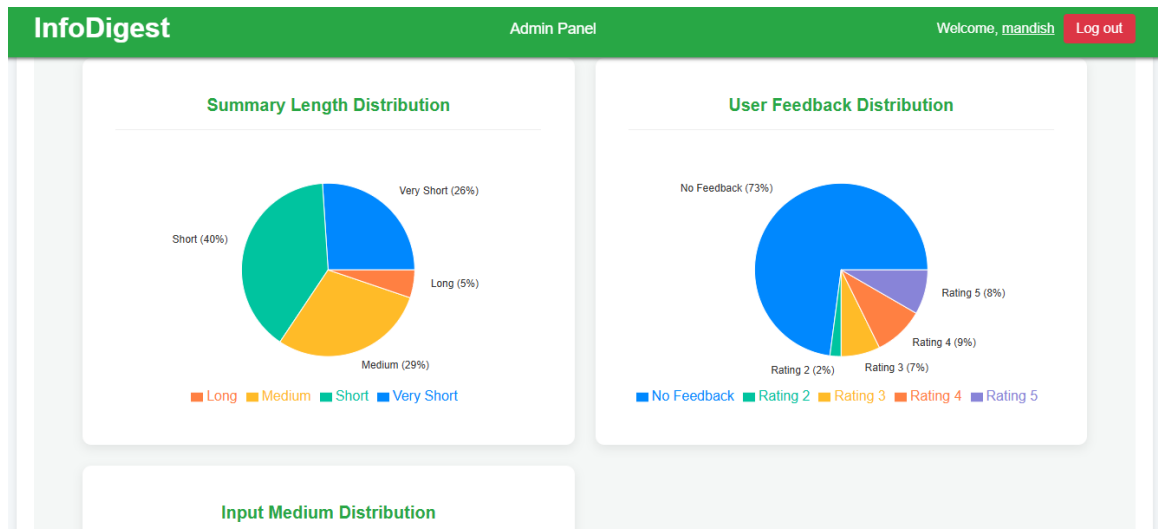


Figure 5.16: Analytics module - 2

5.2. Testing

5.2.1. Test Cases for Unit Testing

Unit testing is a software development process where the smallest testable parts of an application, called units, are individually and independently analyzed for proper operation. The primary purpose is to validate that each unit of the software performs as designed and to quickly detect bugs early in the development cycle.

Test scenario: To check if login module is working correctly.

Table 5.1: Unit testing for login module

TestID	Test steps	Test data	Expected result	Actual result	Status
UT11	Enter valid username and password. Then press login button.	username: “mandish” password: “mandish123”	The user should log in successfully and be redirected to the summarizer page.	The user logs in successfully and is redirected to the summarizer page.	Pass
UT12	Enter invalid username and password. Then press login button.	username: “ramShyam” password: “ram123”	“User not found” error message should be displayed.	“User not found” error message is displayed.	Pass
UT13	Enter valid username and invalid password. Then press login button.	username: “mandish” password: “123456M”	“Incorrect password” error message should be displayed.	“Incorrect password” error message is displayed.	Pass
UT14	Do not enter anything. Then press login button.	username: “” password: “”	“Username is required” and “Password is required” error message should be displayed.	“Username is required” and “Password is required” error message is displayed.	Pass

Test scenario: To check if summarizer module is working correctly.

Table 5.2: Unit testing for summarizer module

TestID	Test steps	Test data	Expected result	Actual result	Status
UT21	Enter a long sentence for text input type. Then press summarize button.	text: “InfoDigest is an AI-powered web application designed...”	The summary and its metrics should be displayed.	The summary and its metrics are displayed.	Pass
UT22	Do not enter anything. Then press summarize button.	text: “”	“Error: Please enter some text first.” error message should be displayed.	“Error: Please enter some text first.” error message is displayed.	Pass
UT23	Enter a long sentence for text input and short summary length. Then press summarize button.	text: “InfoDigest is an AI-powered web application...” summary length: “short”	A short summary and its metrics should be displayed.	A short summary and its metrics are displayed.	Pass
UT24	Select file input. Upload document. Then press summarize button.	file: “final.docx”	The summary and its metrics should be displayed.	The summary and its metrics are displayed.	Pass

5.2.2. Test Cases for System Testing

System testing is a type of software testing that is conducted on a complete, integrated system to evaluate the system's compliance with its specified requirements. The purpose of this testing is to validate the end-to-end functionality of the application in an environment that is as close to production as possible.

Table 5.3: System testing

TestID	Test steps	Test data	Expected result	Actual result	Status
ST1	Enter a long sentence for text input type. Then press summarize button.	text: “InfoDigest is an AI-powered web application designed...” summaryType: “extractive”	The extractive summary and its metrics should be displayed.	The extractive summary and its metrics are displayed.	Pass
ST2	Enter a long sentence for text input type. Then press summarize button.	text: “InfoDigest is an AI-powered web application designed...” summaryType: “abstractive”	The abstractive summary and its metrics should be displayed.	The abstractive summary and its metrics are displayed.	Pass
ST3	Select file input. Upload document. Then press summarize button.	file: “final.docx”	The summary and its metrics should be displayed.	The summary and its metrics are displayed.	Pass

ST4	Enter valid username and password. Then press login button.	username: "mandish" password: "mandish123"	The user should log in successfully and be redirected to the summarizer page.	The user logs in successfully and is redirected to the summarizer page.	Pass
ST5	After summarization, select a feedback option. Then press submit feedback button.	option: 4	"Summary feedback updated successfully!" pop up should be displayed.	"Summary feedback updated successfully!" pop up is displayed.	Pass
ST6	Open user profile by clicking on username on status bar.	username: "mandish"	The user profile of the current user should be displayed.	The user profile of the current user is displayed.	Pass

5.3. Result Analysis

Result analysis for InfoDigest, would involve a systematic evaluation of the effectiveness of both extractive and abstractive summarization methods. This analysis compares the quality, coherence, and accuracy of the summaries generated by each approach.

5.3.1 Extractive Summarization Analysis:

We evaluated the extractive summarization system on a subset of 200 documents from the CNN/DailyMail (3.0.0) test set. The system achieved the following ROUGE F1 scores:

- ROUGE-1: 0.2803
- ROUGE-2: 0.0903
- ROUGE-L: 0.1876

These metrics provide an indication of how closely the generated summaries overlap with human-written reference highlights. ROUGE-1 captures unigram overlap, measuring basic word coverage. ROUGE-2 considers bigram overlap, reflecting the preservation of short phrases and local coherence. ROUGE-L evaluates the longest common subsequence, which reflects sentence-level structure and ordering similarity.

For context, we compare our results against well-known baselines:

Lead-3 Baseline (reported):

- ROUGE-1 $\approx 0.40\text{--}0.43$
- ROUGE-2 $\approx 0.17\text{--}0.20$
- ROUGE-L ≈ 0.36

TextRank (reported in prior work):

- ROUGE-1 $\approx 0.30\text{--}0.33$
- ROUGE-2 $\approx 0.11\text{--}0.13$
- ROUGE-L $\approx 0.25\text{--}0.28$

Interpretation

The system’s performance is below the Lead-3 baseline, which is a strong heuristic for news summarization since the most important information often appears in the first three sentences of articles. Compared to published TextRank baselines, the system’s scores are slightly lower across all metrics. Specifically, the ROUGE-1 and ROUGE-2 scores are close to but below the expected range, while ROUGE-L is noticeably lower. This suggests

that while the system can capture some relevant content, it struggles to consistently align with the reference summaries in terms of structure and sentence selection.

Overall, the evaluation shows that the extractive summarization system produces reasonable results but does not yet match the performance of strong baselines such as Lead-3 or standard TextRank implementations.

5.3.2 Abstractive Summarization Analysis:

We evaluated the LongT5 model on a subset of 100 documents from the arXiv test set. The evaluation used ROUGE metrics to measure the overlap between generated summaries and the reference abstracts. The system achieved the following average ROUGE F1 scores:

- ROUGE-1: 0.4118
- ROUGE-2: 0.1619
- ROUGE-L: 0.2431

These metrics reflect different aspects of summary quality. ROUGE-1 captures unigram overlap, indicating basic content coverage. ROUGE-2 measures bigram overlap, assessing the model’s ability to preserve short phrases and fluency. ROUGE-L evaluates the longest common subsequence, reflecting structural and sentence-level similarity to the reference.

For abstractive summarization on scientific articles (arXiv dataset), the reported baselines typically achieve:

- ROUGE-1: 0.41–0.44
- ROUGE-2: 0.15–0.18
- ROUGE-L: 0.24–0.26

Our model’s performance aligns well with these ranges, indicating that it generates summaries that capture the essential content and structure of scientific abstracts. The ROUGE-1 score of 0.4118 shows that a substantial portion of key terms and concepts is preserved. ROUGE-2 (0.1619) reflects the model’s ability to maintain coherent phrase-level content, while ROUGE-L (0.2431) suggests moderate preservation of sentence-level structure.

Interpretation

The results demonstrate that the LongT5 model effectively generates abstractive summaries that are informative and reasonably faithful to the reference abstracts. The ROUGE scores

indicate that the model captures the main points of each scientific paper while maintaining coherence and grammatical correctness. Some loss in ROUGE-L relative to ROUGE-1 suggests minor differences in sentence structure between the generated and reference summaries, which is expected in abstractive summarization.

Overall, the LongT5 model shows strong performance for abstractive summarization of long scientific documents. The results are consistent with published benchmarks and demonstrate the model's ability to produce concise and informative abstracts. Future improvements could include fine-tuning on domain-specific subsets, increasing beam search size for better content coverage, or experimenting with length constraints to optimize summary conciseness.

CHAPTER 6:

CONCLUSION AND FUTURE RECOMMENDATIONS

6.1. Conclusion

InfoDigest is a powerful web application that effectively tackles the problem of information overload in research and scientific fields. By using advanced AI to create concise summaries of large texts ranging from research papers to standard documents, it makes information more manageable. The system's ability to create both extractive and abstractive summaries is built on a strong foundation, often powered by state-of-the-art models like LongT5.

More than just a summarizer, InfoDigest offers a complete solution with a simple interface, secure user login, and features like keyword extraction and a history of past summaries. This makes it a comprehensive tool for knowledge management. Its well-designed structure also makes it easy to integrate future upgrades, such as RAG models, to further improve the quality of its summaries. Ultimately, InfoDigest is a valuable tool that not only automates a tedious process but also helps researchers more quickly access and use key information.

6.2. Future Recommendations

Looking ahead, the future development of InfoDigest should focus on evolving it from a powerful summarization tool into a more comprehensive and intelligent research assistant. A key recommendation is to integrate a Retrieval-Augmented Generation (RAG) system to enhance summary quality by cross-referencing and enriching the generated content with external, authoritative data. This would allow the platform to produce more factual and contextually rich summaries. Furthermore, the summarization models could be fine-tuned on domain-specific datasets (e.g., medical, legal, or financial) to improve accuracy and relevance for specialized users. To expand its utility, InfoDigest should also pursue integrations with major research databases like PubMed or arXiv, enabling direct summarization of scholarly articles without the need for manual file uploads. Finally, incorporating advanced features such as collaborative annotation and a personalized "research feed" could transform InfoDigest into a central hub for knowledge management, empowering researchers with a tool that not only processes information but also facilitates discovery and collaboration.

REFERENCES

- [1] Grammarly, "Grammarly AI Writing Tools," [Online]. Available: <https://www.grammarly.com/ai/ai-writing-tools/summarizing-tool>. [Accessed 20 May 2024].
- [2] QuillBot, "QuillBot," [Online]. Available: <https://quillbot.com/summarize>. [Accessed 20 May 2024].
- [3] Scribbr, "Scribbr," [Online]. Available: <https://www.scribbr.com/text-summarizer/>. [Accessed 20 May 2024].
- [4] Summarizer.org, "Summarizer.org," [Online]. Available: <https://www.summarizer.org/>. [Accessed 20 May 2024].
- [5] T. Cao, H. Lee and Y. Choi, "Hybrid Extractive-Abstractive Summarization with Retrieval-Augmented Generation," 2023.
- [6] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser and I. Polosukhin, "Attention Is All You Need," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [7] L. Fred, "Automatic Text Summarization with Machine Learning: An Overview," 2021. [Online]. Available: <https://medium.com/luisfredgs/automatic-text-summarization-with-machine-learning-an-overview-68ded5717a25>.
- [8] B. Sharma, A. Choudhary, M. Ahmad and S. Kharel, "Evaluation of Extractive and Abstractive Approaches for Text Summarization," *Engineering Proceedings*, vol. 59, no. 1, p. 194, 2023.
- [9] M. Anwar, R. Raj and K. Yadav, "Text Summarization using Deep Learning: A Study on Automatic Summarization," 2024. [Online]. Available: https://www.researchgate.net/publication/388159354_Text_Summarization_using_Deep_Learning_A_Study_on_Automatic_Summarization.
- [10] Y. Gong and X. Liu, "Text summarization using Latent Semantic Analysis," 2001.

APPENDICES

Landing page:

The screenshot shows the 'InfoDigest' landing page. The header is green with 'InfoDigest' on the left, 'Admin Panel' in the center, and 'Welcome, mandish' and a red 'Log out' button on the right. The main content area is titled 'Summarization' in green. It features three settings: 'Summary Length' with a slider between 'Short' and 'Long' (set to the middle), 'Summary Type' with 'Extractive' and 'Abstractive' buttons (the latter is highlighted in green), and 'Input Type' with 'Text input' and 'File input' buttons (the latter is highlighted in green). Below these is a large text input field with the placeholder 'Enter text to summarize'. At the bottom is a green 'Summarize' button.

File input type:

This screenshot shows the 'InfoDigest' interface with the 'File input' type selected. The header is identical to the previous screenshot. In the 'Summarization' section, the 'File input' button is highlighted in green. Below the settings, there is a text prompt 'Enter .docx, .txt or .pdf'. A file selection box contains a 'Choose File' button and the text 'No file chosen'. A green 'Summarize' button is at the bottom.

Summary:

InfoDigest

Admin Panel

Welcome, [mandish](#) [Log out](#)

Original Text:

Artificial Intelligence, or AI, is one of the most transformative technologies of our time. It refers to the development of computer systems capable of performing tasks that normally require human intelligence. These tasks include learning, reasoning, problem-solving, perception, language understanding, and even creativity. AI is not a single technology but a broad field encompassing machine learning, neural networks, deep learning, natural language processing, robotics, and more. One of the earliest goals of AI was to create systems that could mimic human cognition. Over the decades, AI has progressed from simple rule-based systems to complex algorithms capable of self-learning. Machine learning, a subfield of AI, allows systems to improve automatically through experience without being explicitly programmed. Deep learning, which uses neural networks with many layers, has made breakthroughs in image recognition, speech processing, and language translation. AI is behind the voice assistants in our smartphones, the recommendation systems in streaming services, and the self-driving cars on the roads.

Original Text Length: 46 sentences
Original word count: 684

Summary:

Artificial Intelligence, or AI, is one of the most transformative technologies of our time. One of the earliest goals of AI was to create systems that could mimic human cognition. Privacy is at risk when AI is used to analyze massive amounts of personal data. AI is even used in space exploration, helping analyze data from distant planets and optimize mission planning. While some worry AI may eventually surpass human intelligence, others argue it will always remain a tool shaped by human values.

Summary Length: 5 sentences
Summary word count: 83
Keywords:

- evolve
- relationship

Summary stored message:

InfoDigest

Admin Panel

Welcome, [mandish](#) [Log out](#)

Original Text:

Artificial Intelligence, or AI, is one of the most transformative technologies of our time. It refers to the development of computer systems capable of performing tasks that normally require human intelligence. These tasks include learning, reasoning, problem-solving, perception, language understanding, and even creativity. AI is not a single technology but a broad field encompassing machine learning, neural networks, deep learning, natural language processing, robotics, and more. One of the earliest goals of AI was to create systems that could mimic human cognition. Over the decades, AI has progressed from simple rule-based systems to complex algorithms capable of self-learning. Machine learning, a subfield of AI, allows systems to improve automatically through experience without being explicitly programmed. Deep learning, which uses neural networks with many layers, has made breakthroughs in image recognition, speech processing, and language translation. AI is behind the voice assistants in our smartphones, the recommendation systems in streaming services, and the self-driving cars on the roads.

Original Text Length: 46 sentences
Original word count: 684

Summary:

One of the earliest goals of AI was to create systems that could mimic human cognition. AI is even used in space exploration, helping analyze data from distant planets and optimize mission planning. While some worry AI may eventually surpass human intelligence, others argue it will always remain a tool shaped by human values.

Summary Length: 3 sentences
Summary word count: 54
Keywords:

- evolve
- relationship
- society

Summary stored successfully!
[OK](#)

Feedback section:

InfoDigest

Admin Panel

Welcome, mandish [Log out](#)

- transformative
- learn
- advance
- becomes
- collaboration
- vital
- access

Rate Your Experience

☐ 1 Very Poor ☐ 2 Poor ☐ 3 Average ☒ 4 Good ☐ 5 Excellent

Submit Feedback

Feedback update message:

InfoDigest

Admin Panel

Welcome, mandish [Log out](#)

- transformative
- learn
- advance
- becomes
- collaboration
- vital

Summary feedback updated successfully!

OK

Rate Your Experience

☐ 1 Very Poor ☐ 2 Poor ☐ 3 Average ☒ 4 Good ☐ 5 Excellent

Submit Feedback

User access modifier:

The screenshot shows the 'All Registered Users' page in the InfoDigest Admin Panel. A modal is open for user 'Nasu', showing the current status as 'Admin' and options to 'Make Admin', 'Make Default User', or 'Cancel'.

Profile Picture	Full Name	Username	Email	Gender	Date of Birth	Admin	Registered On	Actions
	uttu	uttu					7/1/2025	Modify Access
	mandish	mandish					7/1/2025	
	pandu	pandu	pandy@pandu.com	Male	6/9/2025	No	7/2/2025	Modify Access
	Nasu	nasu_mdr	nasu@gmail.com	Female	6/18/2003	Yes	8/28/2025	Modify Access

Registered admins:

The screenshot shows the 'Manage Users' section in the InfoDigest Admin Panel. It displays a table of 'Registered Admins' with columns for Profile Picture, Full Name, Username, Email, Gender, Date of Birth, Admin status, Registered On date, and Actions.

Profile Picture	Full Name	Username	Email	Gender	Date of Birth	Admin	Registered On	Actions
	mandish	mandish	mandishvaidya@gmail.com	Male	3/12/2025	Yes	7/1/2025	
	Nasu	nasu_mdr	nasu@gmail.com	Female	6/18/2003	Yes	8/28/2025	Modify Access